

Linear and Sublinear Time Spectral Density Estimation

Vladimir Braverman
Johns Hopkins University
vova@cs.jhu.edu

Aditya Krishnan
Johns Hopkins University
akrish23@jhu.edu

Christopher Musco
New York University
cmusco@nyu.edu

April 2, 2021

Abstract

We analyze the popular *kernel polynomial method* (KPM) for approximating the spectral density (eigenvalue distribution) of a real symmetric (or Hermitian) matrix $A \in \mathbb{R}^{n \times n}$. We prove that a simple and practical variant of the KPM algorithm can approximate the spectral density to ϵ accuracy in the Wasserstein-1 distance with roughly $O(1/\epsilon)$ matrix-vector multiplications with A . This yields a provable linear time result for the problem.

The KPM variant we study is based on *damped Chebyshev polynomial expansions*. We show that it is stable, meaning that it can be combined with any approximate matrix-vector multiplication algorithm for A . As an application, we develop an $O(n/\text{poly}(\epsilon))$ time algorithm for computing the spectral density of any $n \times n$ normalized graph adjacency or Laplacian matrix. This runtime is sublinear in the size of the matrix, and assumes sample access to the graph.

Our approach leverages several tools from approximation theory, including Jackson’s seminal work on approximation with positive kernels [Jac12], and stability properties of three-term recurrence relations for orthogonal polynomials.

1 Introduction

A ubiquitous task in computational science, engineering, and data science is to extract information about the eigenvalue spectrum of a matrix $A \in \mathbb{R}^{n \times n}$. A full eigendecomposition takes at least $O(n^\omega)$ time¹ [Par98, BVKS19], which is prohibitively expensive for large matrices. So, we are typically interested in extracting *partial information* about the spectrum. This can be done using iterative methods like the power or Lanczos methods, which access A via a small number of matrix-vector multiplications. Each matrix-vector multiply takes at most $O(n^2)$ time to compute, and can be accelerated when A is sparse or structured, leading to fast algorithms.

However, the partial spectral information computed by such methods is limited. They typically only compute approximations to the outlying, or *largest magnitude* eigenvalues of A , missing information about the *interior* of A ’s spectrum that may be critical in applications. In network science, clusters of interior eigenvalues can be used to identify graph structures like repeated motifs [DBB19]. In deep learning, information on how the spectrum of a weight matrix differs from its random initialization can give hints about model convergence and generalization [PSG18, MM19], and Hessian eigenvalues are useful in optimization [GKX19]. Coarse information about interior eigenvalues is also used to initialize parallel GPU based methods for full eigendecomposition [AKS17, LXES19].

¹Here $\omega < 2.373$ is the fast matrix multiplication exponent.

To address these needs and many other applications, there has been substantial interest in methods for estimating the full *spectral density* of a matrix A [WWAF06]. Concretely, assume that A is Hermitian, so has real eigenvalues $\lambda_1, \dots, \lambda_n$ and view its spectrum as a probability density s :

$$\text{Spectral density:} \quad s(x) = \frac{1}{n} \sum_{i=1}^n \delta(x - \lambda_i). \quad (1)$$

Here δ is the Dirac delta function. The goal is to find a probability density q that approximates s in some natural metric, like the Wasserstein distance. The density q can be either continuous (represented in some closed form) or discrete (represented as a list of approximate eigenvalues $\tilde{\lambda}_1, \dots, \tilde{\lambda}_n$). See Figure 1 for an illustration. Both are useful in applications.

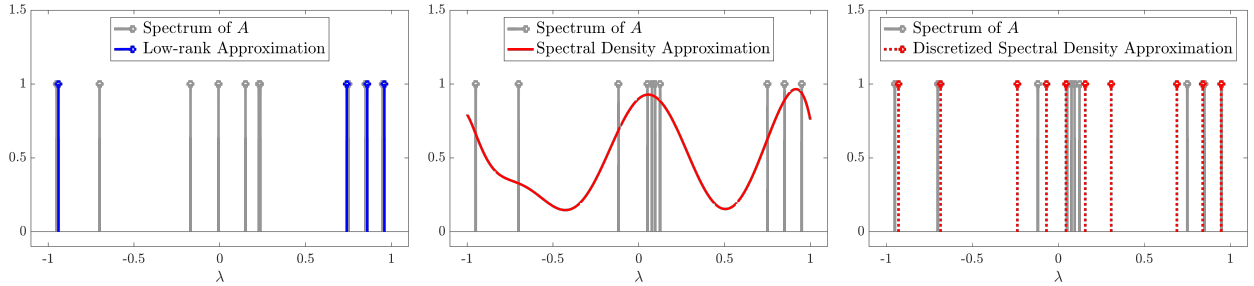


Figure 1: Different approximations for the spectrum of a matrix A with eigenvalues in $[-1, 1]$. A typical approximation computed using an iterative eigenvalue algorithm mostly preserves information about the largest magnitude eigenvalues. In contrast, the spectral density estimates in the two right figures coarsely approximate the entire distribution of A ’s eigenvalues, the first with a low-degree polynomial, and the second with a discrete distribution.

Methods for spectral density estimation that run in $o(n^\omega)$ time were first introduced for applications in condensed matter physics and quantum chemistry [Ski89, SR94, Wan94]. Many are based on the combination of two important tools: 1) moment matching, and 2) stochastic trace estimation. Specifically, if we had access to moments of the distribution s , i.e. $\frac{1}{n} \sum_{i=1}^n \lambda_i$, $\frac{1}{n} \sum_{i=1}^n \lambda_i^2$, $\frac{1}{n} \sum_{i=1}^n \lambda_i^3$, etc., then we could find a good q by finding a distribution that agrees with s on these moments. Moreover, these *spectral moments* can be computed via the matrix trace: note that $\text{tr}(A) = \sum_{i=1}^n \lambda_i$, $\text{tr}(A^2) = \sum_{i=1}^n \lambda_i^2$, $\text{tr}(A^3) = \sum_{i=1}^n \lambda_i^3$, etc. While we cannot hope to compute $\text{tr}(A^k)$ exactly in $o(n^\omega)$ time, thanks to the existence of stochastic trace estimators, like Hutchinson’s method [Hut90, AT11], the trace can be approximated much more quickly. Specifically, stochastic trace estimators are based on the observation that, for any matrix $B \in \mathbb{R}^{n \times n}$, $\text{tr}(B)$ can be well approximated by $\text{tr}(G^T B G)$ where $G \in \mathbb{R}^{n \times m}$ contains random sub-Gaussian entries and $m \ll n$. For any k degree polynomial g , $G^T g(A) G$ can be computed with just $O(km)$ matrix-vector multiplications, so we can quickly approximate any low-degree moment of A ’s spectral density.

This high-level approach and related techniques have been applied successfully to estimating the spectra of a wide variety of matrices [WWAF06, LSY16]. For example, the popular kernel polynomial method (KPM) is a moment matching approach, based on estimating Chebyshev or Legendre moments of A using Hutchinson’s estimator. However, no worst-case approximation guarantees have been proven for such methods, making it challenging to predict their performance on general inputs and to theoretically compare different algorithmic variants.

1.1 Our contributions

The goal of this paper is to develop strong theoretical guarantees for variants of the kernel polynomial method, which is one of the most popular spectrum estimation algorithm used in practice. We target approximation in the Wasserstein-1 metric, also known as earthmover’s distance. This metric is one of the most natural for comparing a discrete distribution to a possibly continuous one (like a spectral density estimate). Given a matrix A with spectral density s , our goal is to return some representation of a density q such that:

$$W_1(s, q) \leq \epsilon. \quad (2)$$

We defer a formal definition of W_1 to Section 2. This metric has been recently used in analyzing spectral density estimation algorithms for graphs [DBB19, CKSV18]. A nice property is that, if we let $\Lambda = [\lambda_1, \dots, \lambda_n]$ be a vector of A ’s eigenvalues, then if $\tilde{\Lambda} = [\tilde{\lambda}_1, \dots, \tilde{\lambda}_n]$ satisfies $\|\Lambda - \tilde{\Lambda}\|_1 \leq n\epsilon$, we have that the discrete spectral density q with eigenvalues in $\tilde{\Lambda}$ satisfies $W_1(s, q) \leq \epsilon$.

Linear time spectral density estimation

Our first result holds for any Hermitian matrix A (e.g., real symmetric), and is stated under the assumption that $\|A\|_2 \leq 1$, i.e., that A ’s eigenvalues lie in $[-1, 1]$. It can be applied to any matrix A after properly scaling by $1/\|A\|_2$.

Theorem 1.1 (Linear time spectral density estimation). *Let $A \in \mathbb{R}^{n \times n}$ be a Hermitian matrix with spectral density s and $\|A\|_2 \leq 1$. Let C, C' be fixed positive constants. For any $\epsilon, \delta \in (0, 1)$, there is a variant of the kernel polynomial method (Algorithm 2, with Algorithm 3 used as a subroutine to approximate moments) which computes $T = C\ell/\epsilon$ matrix-vector multiplications with A where $\ell = \max\left(1, \frac{C'}{n}\epsilon^{-4} \log^2\left(\frac{1}{\epsilon\delta}\right)\right)$, and in $O(n\ell/\epsilon)$ additional runtime, outputs a probability density function $q : [-1, 1] \rightarrow \mathbb{R}^{\geq 0}$ such that $W_1(s, q) \leq \epsilon$ with probability $1 - \delta$.*

Theorem 1.1 is proven in Section 5.1. Note that the result actually *improves* as n increases, requiring just $T = O(1/\epsilon)$ matrix vector multiplies when n is sufficiently large compared to $1/\epsilon$. For smaller n , the polynomial dependence on ϵ is worse. Intuitively, when A is larger, each matrix-vector product returns more information about the spectral density s , so we can estimate it more easily. The runtime of Theorem 1.1 is dominated by the cost of the matrix-vector multiplications, which take $O(T \cdot n^2)$ time to compute for a dense matrix, and $O(T \cdot \text{nnz}(A))$ time for a sparse matrix with $\text{nnz}(A)$ non-zero entries, so the algorithm runs in linear time when ϵ, δ are considered constant. The density function q returned by Algorithm 2 is in the form of a low-degree (specifically $O(1/\epsilon)$ degree) polynomial, multiplied by a simple closed form function. It is easy to plot, integrate, or manipulate. Alternatively, a simple rounding scheme due to [CKSV18] which runs in $O(n + \text{poly}(1/\epsilon))$ time can extract from q a vector of approximate eigenvalues $\tilde{\Lambda} = [\tilde{\lambda}_1, \dots, \tilde{\lambda}_n]$ satisfying $\|\Lambda - \tilde{\Lambda}\|_1 \leq 2n\epsilon$, which as discussed is 2ϵ close to the spectral density s in Wasserstein distance (see Theorem A.1).

Theorem 1.1 is based on a version of the kernel polynomial method that approximates *Chebyshev polynomial* moments: $\text{tr}(T_0(A)), \dots, \text{tr}(T_N(A))$ where $T_0, \dots, T_N$ are the Chebyshev polynomials of the first kind. We show that, if these moments are computed exactly, we can find a density function q close to s in Wasserstein-1 distance. To do so, we turn to classic work of Jackson on universal polynomial approximation bounds for Lipschitz functions: we take advantage of the fact

that Jackson’s construction of such polynomials is both *linear* and *preserves positivity* [Jac12]. We then prove that it is still possible to extract a spectral density estimation even when the Chebyshev moments are computed to relatively low precision – in contrast to methods based on the standard moments, this allows use to obtain a polynomial dependence on ϵ in Theorem 1.1.

Sublinear time spectral density estimation

Above it is assumed that our spectrum approximation algorithm can compute exact matrix-vector multiplications with A in linear time. However, in practice, spectral density estimation is often applied to matrices where we only have *implicit* access to A , and in many cases, multiplications can only be computed approximately. For example, A might be a Hessian matrix that we can multiply by approximately using Pearlmutter’s trick [Pea94, YGKM20], or the inverse of some other matrix, which we can multiply by approximately using an iterative solver. Such cases motivate our second main result, which only assumes access to an approximate matrix vector multiplication oracle:

Definition 1.2. An ϵ_{MV} -approximate matrix-vector multiplication oracle for $A \in \mathbb{R}^{n \times n}$ and error parameter $\epsilon_{MV} \in (0, 1)$ is an algorithm that, given any vector $y \in \mathbb{R}^n$, outputs a vector z such that $\|z - Ay\|_2 \leq \epsilon_{MV} \|A\|_2 \|y\|_2$. We will denote a call to such an oracle for by $\text{AMV}(A, y, \epsilon_{MV})$.

In Section 5.2 we prove the following robust version of Theorem 1.1:

Theorem 1.3 (Robust spectral density estimation). *Let $A \in \mathbb{R}^{n \times n}$ be a Hermitian matrix with spectral density s and $\|A\|_2 \leq 1$. Let C, C', C'' be fixed positive constants. For any $\epsilon, \delta \in (0, 1)$ and $\epsilon_{MV} = C''\epsilon^{-4}$, there is a variant of the kernel polynomial method (Algorithm 2, with Algorithm 4 used as a subroutine to approximate moments) which makes $T = C\ell/\epsilon$ calls to an ϵ_{MV} -approximate matrix-vector oracle for A , where $\ell = \max\left(1, \frac{C'}{n}\epsilon^{-4} \log^2\left(\frac{1}{\epsilon\delta}\right)\right)$, and in $O(n\ell/\epsilon)$ additional runtime, outputs a probability density function $q : [-1, 1] \rightarrow \mathbb{R}^{\geq 0}$ such that $W_1(s, q) \leq \epsilon$ with probability $1 - \delta$.*

The proof of Theorem 1.3 requires showing that Chebyshev moments can be accurately computed using an approximate matrix-vector oracle. We are able to do so by drawing on stability results for the three-term recurrence relation defining these polynomials [Cle55, MMS18]. As a concrete application of Theorem 1.3, in Section 6 we introduce a random sampling algorithm that can implement an ϵ -approximate matrix-vector multiplication oracle for any *symmetric normalized graph adjacency matrix* A in $O(n/\epsilon)$ time. Combined with Theorem 1.3, this yields:

Theorem 1.4 (Sublinear time spectral density estimation for graphs.). *Let $G = (V, E)$ be an unweighted, undirected n -vertex graph and let $A \in \mathbb{R}^{n \times n}$ be the normalized adjacency of G with spectral density s . Let $\epsilon, \delta \in (0, 1)$ be fixed values. Given access to the degrees of the vertices of G , there is a randomized algorithm with expected running time $O(n \text{ poly}(\log(1/\delta)/\epsilon))$ which outputs a density function $q : [-1, 1] \rightarrow \mathbb{R}^+$ such that $W_1(q, s) \leq \epsilon$ with probability at least $1 - \delta$.*

Since A can have up to n^2 non-zero entries, the runtime above is *sublinear* in the input size, illustrating the power of Theorem 1.3. Note that the normalized graph Laplacian $L = I - A$ has the same eigenvalues as A up to a reflection, so Theorem 1.4 also yields a sublinear time result for normalized Laplacians, whose spectral densities are of interest in network science [DBB19].

1.2 Related work

Spectral density estimation. As discussed, algorithms for spectral density estimation have been studied since the early 90s [Ski89, SR94, Wan94]. We refer the reader to the excellent survey by [WWAF06]. The Wasserstein-1 distance has been suggested as a natural metric for evaluating the error of spectral density estimates (e.g. in [DBB19]). While we are not aware of any direct results providing a linear time algorithm with a worst-case guarantee, as in Theorem 1.1, it is possible to obtain a similar result from the work of Valiant and Kong on learning a distribution in Wasserstein-1 distance based on approximations to the standard moments [KV17]. Specifically, moment estimates for A obtained via Hutchinson’s estimator could be directly plugged into that result to approximate the spectral density. However, such an approach inherently incurs an inverse *exponential* dependence on ϵ , whereas Theorem 1.1 has a polynomial dependence.

For the special case of normalized Laplacian matrices, Cohen et al. [CKSV18] prove a result which matches the guarantee of Theorem 1.4, but with runtime of $2^{O(1/\epsilon)}$ – i.e., with *no dependence* on n . In comparison, our result depends linearly on n , but only polynomially on $1/\epsilon$. An interesting open question is if a $1/\text{poly}(\epsilon)$ time algorithm is possible.

Matrix-vector query algorithms. Our work fits into a broader line of work on proving upper and lower bounds on the *matrix-vector query complexity* of linear algebraic problems, from top eigenvector, to matrix inversion, to rank estimation [SWYZ19, SEAR18, BHSW20, MMMW20, DM21]. The goal in this model is to minimize the total number of matrix-vector multiplications with A , recognizing that such multiplications either 1) dominate runtime cost or 2) are the only way to access A when it is an implicit matrix. The matrix-vector query model generalizes both classical Krylov subspace methods, as well as randomized sketching methods [Woo14]. Studying other basic linear algebra problem when matrix-vector multiplication queries are only assumed to be approximate (as in Definition 1.2) is an interesting future direction.

1.3 Paper Roadmap

We describe notation and preliminaries on polynomial approximation in Section 2. We use these tools in Section 3 and Section 4 to prove that a good approximation to the first $O(1/\epsilon)$ Chebyshev polynomial moments of the spectral density can be used to extract a good approximation in Wasserstein-1 distance. This result is the basis for our main results, Theorem 1.1 and Theorem 1.3, which are proven in Section 5. Finally, we prove Theorem 1.4 in Section 6.

2 Preliminaries

Throughout we assume that $A \in \mathbb{R}^{n \times n}$ is Hermitian with eigendecomposition $A = U\Lambda U^*$, where $UU^* = U^*U = I_{n \times n}$. We assume that A ’s eigenvalues satisfy $-1 \leq \lambda_n \leq \dots \leq \lambda_1 \leq 1$. In many applications A is real symmetric. We denote A ’s spectral density by s , which is defined in (1). Our goal is to approximate s in the Wasserstein-1 metric with another distribution q supported on $[-1, 1]$. Specifically, as per the dual formulation given by the Kantorovich-Rubinstein theorem

[KR57], for s, q supported on $[-1, 1]$ the metric is equal to:

$$W_1(s, q) = \sup_{\substack{f: \mathbb{R} \rightarrow \mathbb{R} \\ |f(x) - f(y)| \leq |x - y| \quad \forall x, y}} \left\{ \int_{-1}^1 f(x) (s(x) - q(x)) dx \right\}. \quad (3)$$

In words, s and q are close in Wasserstein-1 distance if their difference has small inner product with all 1-Lipschitz functions f . Alternatively, $W_1(s, q)$ is equal to the cost of “changing” one distribution to another, where the cost of moving one unit of mass from x to y is $|x - y|$: this is the “earthmover’s” formulation common in computer science. Note that (3) can be applied to arbitrary functions s, q , even if they are *not distributions*, and we will occasionally do so.

Functions and inner products. We introduce notation for functions used throughout the paper. Let $\mathcal{F}([-1, 1], \mathbb{R})$ denote the space of real-valued functions on $[-1, 1]$. For $g, h \in \mathcal{F}([-1, 1], \mathbb{R})$, let $\langle g, h \rangle$ denote $\langle g, h \rangle := \int_{-1}^1 g(x)h(x)dx$. For $f \in \mathcal{F}([-1, 1], \mathbb{R})$, we define $\|f\|_2 := \sqrt{\langle f, f \rangle}$ and let $\|f\|_\infty$ denote the max-norm $\|f\|_\infty = \max_{x \in [-1, 1]} |f(x)|$. We let $\|f\|_1$ denote $\|f\|_1 = \int_{-1}^1 |f(x)|dx$.

Let $\mathcal{F}(\mathbb{Z}, \mathbb{R})$ be the space of real-valued functions on the integers, $\mathbb{Z}$. For $f, g \in \mathcal{F}(\mathbb{Z}, \mathbb{R})$ let $(f * g)$ denote the discrete convolution: $(f * g)[n] = \sum_{m=-\infty}^{\infty} f[m]g[n - m]$. Let $\mathcal{F}(\mathbb{N}, \mathbb{R})$ be the space of real-valued functions on the natural numbers, $\mathbb{N}$. For functions in $\mathcal{F}(\mathbb{Z}, \mathbb{R})$ or $\mathcal{F}(\mathbb{N}, \mathbb{R})$ we typically used square brackets instead of parentheses.

For two functions f, g let $h = fg$ (or $h = f \cdot g$) and $j = f/g$ denote the pointwise product and quotient respectively. I.e. $h(x) = f(x)g(x)$ and $j(x) = f(x)/g(x)$ for all x .

Chebyshev polynomials. Our approach is based on approximating Chebyshev polynomial moments of A ’s spectral density, and we will use basic properties of these polynomials, the k^{th} of which we denote T_k . The Chebyshev polynomial of the first kind can be defined via the recurrence:

$$\begin{aligned} T_0(x) &= 1 & T_1(x) &= x \\ T_k(x) &= 2x \cdot T_{k-1}(x) - T_{k-2}(x) & \text{for } k \geq 2. \end{aligned}$$

We will use the well known fact that the Chebyshev polynomials of the first kind are bounded between $[-1, 1]$, i.e. $\max_{x \in [-1, 1]} |T_k(x)| \leq 1$.

Let $w(x) := \frac{1}{\sqrt{1-x^2}}$. It is well known that $\langle T_0, w \cdot T_0 \rangle = \pi$, $\langle T_k, w \cdot T_k \rangle = \pi/2$ for $k > 0$, and

$$\langle T_i, w \cdot T_j \rangle = 0 \quad \text{for } i \neq j.$$

In other words, the Chebyshev polynomials are orthogonal on $[-1, 1]$ under the weight function w . The first k Chebyshev polynomials form an orthogonal basis for the degree k polynomials under this weight function. We let $\bar{T}_k$ denote the *normalized* Chebyshev polynomial $\bar{T}_k := T_k / \sqrt{\langle T_k, w \cdot T_k \rangle}$.

Definition 2.1 (Chebyshev Series). The *Chebyshev expansion or series* for a function $f \in \mathcal{F}([-1, 1], \mathbb{R})$ is given by

$$\sum_{k=0}^{\infty} \langle f, w \cdot \bar{T}_k \rangle \cdot \bar{T}_k.$$

We call $\sum_{k=0}^N \langle f, w \cdot \bar{T}_k \rangle \cdot \bar{T}_k$ the truncated Chebyshev expansion or series of degree N .

Other notation. Let $[n]$ denote $1, \dots, n$. For a scalar function $f : \mathbb{R} \rightarrow \mathbb{R}$ and $n \times n$ matrix A with eigendecomposition $U\Lambda U^*$, we let $f(A)$ denote the matrix function $Uf(\Lambda)U^*$. Here $f(\Lambda)$ is understood to mean f applied entrywise to the diagonal matrix Λ containing A 's eigenvalues. Note that $\text{tr}(f(A)) = \sum_{i=1}^n f(\lambda_i)$. When $f(x)$ is a degree q polynomial, $c_0 + c_1x + \dots, c_qx^q$, then we can check that $f(A)$ exactly equals $c_0I + c_1A + \dots, c_qA^q$, where I is then $n \times n$ identity matrix. So $f(A)y$ can be computed for any vector y using q matrix-vector multiplications with A .

3 Idealized Kernel Polynomial Method

In this section we show that the spectral density s of a matrix A with eigenvalues in $[-1, 1]$ can be well approximated given access to the first $N = O(1/\epsilon)$ Chebyshev polynomial moments of s , i.e., to $\text{tr}(\bar{T}_0(A)), \dots, \text{tr}(\bar{T}_N(A))$. This result will form the basis for the full kernel polynomial method we analyze in Section 4, which approximates these moments using a stochastic trace estimator, implemented with either exact or approximate matrix vector multiplications with A .

A natural approach is to use these moments to construct a truncated Chebyshev series approximation to s (see Definition 2.1). To do so, note that the scaled moments $\frac{1}{n} \text{tr}(\bar{T}_0(A)), \dots, \frac{1}{n} \text{tr}(\bar{T}_N(A))$ are exactly equal to the first N Chebyshev series coefficients of s/w , where $w(x) = \frac{1}{\sqrt{1-x^2}}$ is as defined in Section 2. Specifically, the eigenvalues of $\bar{T}_k(A)$ are equal to $\bar{T}_k(\lambda_1), \dots, \bar{T}_k(\lambda_n)$, where $\lambda_1, \dots, \lambda_n$ are the eigenvalues of A . Since the trace of a diagonalizable matrix is the sum of its eigenvalues, we have $\frac{1}{n} \text{tr}(\bar{T}_k(A)) = \frac{1}{n} \sum_{i=1}^n \bar{T}_k(\lambda_i) = \langle s, \bar{T}_k \rangle = \langle s/w, w \cdot \bar{T}_k \rangle$.

After using the scaled Chebyshev moments to construct a truncated Chebyshev series for s/w , i.e. a degree N polynomial approximation, we can then multiply the final result by w to obtain an approximation to s . Unfortunately, there are two issues with this approach: 1) it is difficult to analyze the quality of the Chebyshev series approximation, since s is not a smooth function, and 2) this approximation will not in general be a non-negative function, which is a challenge because our goal is to find *probability density* that well approximates s .

A common approach for dealing with the second issue is to instead use a *damped Chebyshev expansion* [WWAF06], where the Chebyshev coefficients are slightly reweighted to ensure that the resulting polynomial is always non-negative. Such non-negativity preserving damping schemes follow from the connection between Chebyshev and Fourier series: we refer the reader to Appendix B for details. In short, by the convolution theorem, Fourier series truncation corresponds to convolution with a function whose Fourier support is bounded.

If this function is also non-negative, convolution preserves non-negativity of the function being approximated, leading to truncated series that is guaranteed to be positive. One such damping schemes was introduced in classic work of Jackson [Jac12]. For any positive integer z , let $N = 4z$. Then, for $k = 0, \dots, N$, define the coefficient

$$\hat{b}_N[k] = \sum_{j=-\frac{N}{2}-1}^{\frac{N}{2}+1-k} \left(\frac{N}{2} + 1 - |j| \right) \cdot \left(\frac{N}{2} + 1 - |j+k| \right). \quad (4)$$

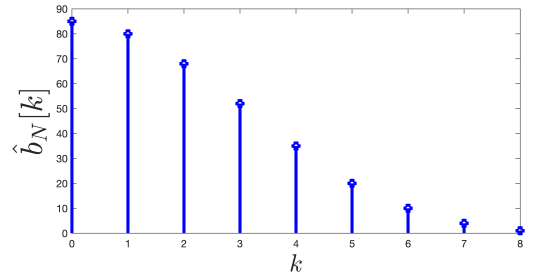


Figure 2: Jackson coefficients for $N = 8$.

While (4) may look opaque, $\hat{b}_N[0], \dots, \hat{b}_N[N]$ are actually equal to the result of a simple discrete convolution operation. Let $g \in \mathcal{F}(\mathbb{Z}, \mathbb{R})$ have $g[j] = 1$ for $j = -z, \dots, z$, and $g[j] = 0$ otherwise. Then let $\hat{b}_N = (g * g) * (g * g)$ and $\hat{b}_N[0], \dots, \hat{b}_N[N]$ be the values corresponding to non-negative indices.² See Fig. 2 for an illustration of these coefficients. They are all positive and $\hat{b}_N[0] > \hat{b}_N[1] > \dots > \hat{b}_N[N]$. Jackson suggests approximating a function using the following truncation based on these coefficients:

Definition 3.1 (Jackson damped Chebyshev series). Let $f \in \mathcal{F}([-1, 1], \mathbb{R})$ have Chebyshev series $\sum_{k=0}^{\infty} \langle f, w \cdot \bar{T}_k \rangle \cdot \bar{T}_k$. The Jackson approximation to f is a degree N polynomial $\bar{f}_N$ obtained via the following truncation with modified coefficients:

$$\bar{f}_N(x) := \sum_{k=0}^N \frac{\hat{b}_N[k]}{\hat{b}_N[0]} \langle f, w \cdot \bar{T}_k \rangle \bar{T}_k(x). \quad (5)$$

Note that $\hat{b}_N[0]/\hat{b}_N[0] = 1$, and all other terms are strictly less than one. It is not hard to show this damped series preserves positivity. We prove the following fact as Lemma B.7 in the appendix:

Fact 3.2. *If $f : [-1, 1] \rightarrow \mathbb{R}^{\geq 0}$ is a non-negative function, then the polynomial $\bar{f}_N(x)$ defined in (5) is non-negative for all $x \in [-1, 1]$.*

Beyond preserving non-negativity, the Jackson damped Chebyshev approximation is more well-known for the fact that it provably provides a good *uniform* polynomial approximation to any Lipschitz function. In fact, an analysis of (5) is how Jackson obtained his famous result on the approximation of Lipschitz functions [Jac30]. Concretely, we have:

Fact 3.3. *Let $f \in \mathcal{F}([-1, 1], \mathbb{R})$ be a Lipschitz continuous function with Lipschitz constant λ . I.e., $|f(x) - f(y)| \leq \lambda|x - y|$ for all $x, y \in [-1, 1]$. For $N \in 4\mathbb{N}^+$, let $\bar{f}_N$ be the degree N polynomial approximation to f from (5). This approximation satisfies $\|\bar{f}_N - f\|_{\infty} \leq 18 \frac{\lambda}{N}$.*

For completeness, we give a proof of this fact as Theorem B.6 in the appendix. With Facts 3.2 and 3.3 in place, we are ready to introduce the basic kernel polynomial method for approximating the spectral density s – it is identical to the “Jackson Kernel” KPM from [WWAF06]. Recall that, for now, we assume we have access to *exact* Chebyshev moment of the spectral density s for our matrix A . In Section 4 we prove that Algorithm 1 is robust to using approximate moments.

Algorithm 1 Idealized Jackson Damped Kernel Polynomial Method

Input: Symmetric $A \in \mathbb{R}^{n \times n}$ with spectral density $s : [-1, 1] \rightarrow \mathbb{R}^{\geq 0}$, degree $N \in 4\mathbb{N}^+$.

Output: Density function $q : [-1, 1] \rightarrow \mathbb{R}^{\geq 0}$.

- 1: For $k = 0, \dots, N$ compute $\tau_k = \frac{1}{n} \text{tr}(\bar{T}_k(A)) = \langle s, \bar{T}_k \rangle$.
 - 2: For $k = 0, \dots, N$ compute $\hat{b}_N[k]$ as is (4).
 - 3: Return $q = w \cdot \sum_{k=0}^N \frac{\hat{b}_N[k]}{\hat{b}_N[0]} \cdot \tau_k \cdot \bar{T}_k$.
-

Lemma 3.4. *If $N \geq \frac{18}{\epsilon}$, then the function $q : [-1, 1] \rightarrow \mathbb{R}^{\geq 0}$ returned by Algorithm 1 is a probability density and satisfies:*

$$W_1(s, q) \leq \epsilon.$$

²This formulation allows the coefficients to be easily computed in most high-level programming languages. E.g., in MATLAB we can compute `g = ones(2*z+1,1); c = conv(conv(g,g),conv(g,g)); b = c(N+1:2*N+1);`.

Proof. We first prove that q is a probability density. To see that it is positive, note that $h = \sum_{k=0}^N \frac{\hat{b}_N[k]}{\hat{b}_N[0]} \cdot \tau_k \cdot \bar{T}_k$ is a Jackson approximation to the positive function s/w , so it must be non-negative by Fact 3.2. Since w is also non-negative, we conclude that $q = w \cdot h$ is as well. Then we consider q 's integral. We need to show that $\int_{-1}^1 q(x)dx = 1 = \int_{-1}^1 s(x)dx$. Since $\bar{T}_0$ is a scaling of the constant function, it suffices to show that $\langle \bar{T}_0, q \rangle = \langle \bar{T}_0, s \rangle$. We have:

$$\langle \bar{T}_0, q \rangle = \tau_0 \cdot \langle \bar{T}_0, w \cdot \bar{T}_0 \rangle = \langle \bar{T}_0, s \rangle \cdot 1.$$

The first step follows directly from the orthogonality of the Chebyshev polynomials under the weight function w , which implies that $\langle \bar{T}_0, w \cdot \bar{T}_k \rangle = 0$ for all $k > 0$. We also use that $\langle \bar{T}_0, w \cdot \bar{T}_0 \rangle = 1$.

Next, we prove the approximation guarantee. Referring to the formulation of Wasserstein-1 distance from equation (3), we have that $W_1(s, q) = \sup \langle s - q, f \rangle$ where f is a 1-Lipschitz function. So, we want to show that any 1-Lipschitz f has small inner product with the difference between s and its degree- N Jackson approximation, q . To do so, we show that this inner product is actually *exactly equal* to the inner product between s and a degree- N Jackson approximation to f . Since f is 1-Lipschitz, this approximation is guaranteed to have small error. This key equivalency follows because, like a standard Chebyshev series approximation, the Jackson approximation can be viewed as the output of a symmetric linear operator applied to s .

Formally, we introduce notation for several linear operators needed to analyze (5). Let $\bar{\mathcal{T}} : \mathcal{F}([-1, 1], \mathbb{R}) \rightarrow \mathcal{F}(\mathbb{N}, \mathbb{R})$ be the operator mapping a function $f \in \mathcal{F}([-1, 1], \mathbb{R})$ to its inner-product with the normalized Chebyshev polynomials. Define the transpose operator $\bar{\mathcal{T}}^* : \mathcal{F}(\mathbb{N}, \mathbb{R}) \rightarrow \mathcal{F}([-1, 1], \mathbb{R})$ to satisfy $\langle \bar{\mathcal{T}}f, g \rangle = \langle f, \bar{\mathcal{T}}^*g \rangle$ for any $g \in \mathcal{F}(\mathbb{N}, \mathbb{R})$. Concretely, for $i \in \mathbb{N}$ and $x \in [-1, 1]$,

$$[\bar{\mathcal{T}}f][i] := \int_{-1}^1 \bar{T}_i(x)f(x)dx \quad \text{and} \quad [\bar{\mathcal{T}}^*g](x) := \sum_{i=0}^{\infty} \bar{T}_i(x)g[i]. \quad (6)$$

We also define operators $\mathcal{W} : \mathcal{F}([-1, 1], \mathbb{R}) \rightarrow \mathcal{F}([-1, 1], \mathbb{R})$ and $\mathcal{I} : \mathcal{F}(\mathbb{N}, \mathbb{R}) \rightarrow \mathcal{F}(\mathbb{N}, \mathbb{R})$ as follows:

$$[\mathcal{W}f](x) := w(x)f(x) = \frac{1}{\sqrt{1-x^2}}f(x) \quad \text{and} \quad [\mathcal{I}g][i] := g[i].$$

Note that $\mathcal{I}$ is an identity operator. For any $N \in 4\mathbb{N}$, we define $\mathcal{B}_N : \mathcal{F}(\mathbb{N}, \mathbb{R}) \rightarrow \mathcal{F}(\mathbb{N}, \mathbb{R})$ as:

$$[\mathcal{B}_N g](i) := \begin{cases} \frac{\hat{\beta}_N[i]}{\hat{\beta}_N[0]}g(i) & \text{for } 0 \leq i \leq N \\ 0 & i > N. \end{cases}$$

The operators $\mathcal{W}$, $\mathcal{I}$, and $\mathcal{B}_N$ are all commutative with respect to the inner-products in their respective spaces. Specifically, for $f_1, f_2 \in \mathcal{F}([-1, 1], \mathbb{R})$ and $g_1, g_2 \in \mathcal{F}(\mathbb{N}, \mathbb{R})$, $\langle f_1, \mathcal{W}f_2 \rangle = \langle \mathcal{W}f_1, f_2 \rangle$, $\langle g_1, \mathcal{I}g_2 \rangle = \langle \mathcal{I}g_1, g_2 \rangle$, and $\langle g_1, \mathcal{B}_N g_2 \rangle = \langle \mathcal{B}_N g_1, g_2 \rangle$. Also note that by orthogonality of the Chebyshev polynomials under w , $\bar{\mathcal{T}}^* \bar{\mathcal{T}} \mathcal{W}$ is the identity operator on $\mathcal{F}([-1, 1], \mathbb{R})$ and so is $\mathcal{W} \bar{\mathcal{T}}^* \bar{\mathcal{T}}$.

With these operators defined, the remainder of the proof is short. We have via direct calculation:

$$\begin{aligned} \langle f, s - q \rangle &= \langle f, s - \mathcal{W} \bar{\mathcal{T}}^* \mathcal{B}_N \bar{\mathcal{T}} s \rangle \\ &= \langle f, \mathcal{W} \bar{\mathcal{T}}^* (\mathcal{I} - \mathcal{B}_N) \bar{\mathcal{T}} s \rangle = \langle \bar{\mathcal{T}}^* (\mathcal{I} - \mathcal{B}_N) \bar{\mathcal{T}} \mathcal{W} f, s \rangle = \langle f - \bar{\mathcal{T}}^* \mathcal{B}_N \bar{\mathcal{T}} \mathcal{W} f, s \rangle. \end{aligned}$$

Note that $\bar{\mathcal{T}}^* \mathcal{B}_N \bar{\mathcal{T}} \mathcal{W} f$ is exactly the degree- N Jackson approximation to f . So by Fact 3.3, if f is a 1-Lipschitz function, $\|f - \bar{\mathcal{T}}^* \mathcal{B}_N \bar{\mathcal{T}} \mathcal{W} f\|_{\infty} \leq 18/N$. Since s is a non-negative function that integrates to 1, it follows that $\langle f, s - q \rangle = \langle f - \bar{\mathcal{T}}^* \mathcal{B}_N \bar{\mathcal{T}} \mathcal{W} f, s \rangle \leq 18/N$. Since $W(s, q) = \sup_{1\text{-Lipschitz } f} \langle f, s - q \rangle$, we conclude that $W(s, q) \leq \epsilon$ as long as $N \geq 18/\epsilon$. $\square$

Remark. Given access to the Chebyshev polynomial moments, $\text{tr}(\bar{T}_0(A)), \dots, \text{tr}(\bar{T}_N(A))$, Algorithm 1 can be implemented in $O(1/\epsilon)$ additional time. The function it returns is an $O(1/\epsilon)$ degree polynomial times the closed form function w . The polynomial can be represented as a sum of Chebyshev polynomials, or converted to standard monomial form in $O(1/\epsilon^2)$ time. The function is easily plotted or integrated over a range – see discussion around Fact A.2 for more details.

4 Full Kernel Polynomial Method

In reality, it is not possible to efficiently compute the exact Chebyshev polynomial moments, $\text{tr}(\bar{T}_0(A)), \dots, \text{tr}(\bar{T}_N(A))$. To obtain the linear and sublinear time results of Theorems 1.1 and Theorems 1.4, we need to work with approximations to these moments, which can be computed using a stochastic trace estimator. We introduce the details of how to do so in Section 5. Here, we first prove a general result on the accuracy of approximation needed to ensure we obtain a good spectral density estimation. Specifically, we analyze the following “robust” version of Algorithm 1.

Algorithm 2 Jackson Damped Kernel Polynomial Method

Input: Symmetric $A \in \mathbb{R}^{n \times n}$ with spectral density $s : [-1, 1] \rightarrow \mathbb{R}^{\geq 0}$, degree parameter $N \in 4\mathbb{N}^+$, algorithm $\mathcal{M}(A)$ that computes moment approximations $\tilde{\tau}_1, \dots, \tilde{\tau}_N$ with the guarantee that $|\tilde{\tau}_k - \frac{1}{n} \text{tr}(\bar{T}_k(A))| \leq 1/N^2$ for all k .

Output: Density function $q : [-1, 1] \rightarrow \mathbb{R}^{\geq 0}$.

- 1: For $k = 1, \dots, N$ use $\mathcal{M}$ to compute $\tilde{\tau}_1, \dots, \tilde{\tau}_N$ as above. Set $\tilde{\tau}_0 = 1/\sqrt{\pi}$.
 - 2: For $k = 0, \dots, N$ compute $\hat{b}_N[k]$ as is (4).
 - 3: Compute polynomial $\tilde{s}_N = w \cdot \sum_{k=0}^N \frac{\hat{b}_N[k]}{\hat{b}_N[0]} \cdot \tilde{\tau}_k \cdot \bar{T}_k$.
 - 4: Return the probability density $q = \left(\tilde{s}_N + \frac{w\sqrt{2}}{N\sqrt{\pi}} \right) / \left(1 + \frac{\sqrt{2\pi}}{N} \right)$.
-

The final transformation of $\tilde{s}_N$ in Line 4 of Algorithm 2 ensures that we return a proper density, since error incurred by approximating $\frac{1}{n} \text{tr}(\bar{T}_k(A)) = \langle s, \bar{T}_k \rangle$ could leave the function with negative values. So, we shift by a small positive function, and rescale to maintain unit integral. Our main result on the error of Algorithm 2, which parallels Lemma 3.4 for Algorithm 1, is as follows:

Lemma 4.1. *If $N \geq \frac{18}{\epsilon}$, then the function $q : [-1, 1] \rightarrow \mathbb{R}^{\geq 0}$ returned by Algorithm 2 is a probability density and satisfies:*

$$W_1(s, q) \leq 2\epsilon.$$

Remark. When $N = O(1/\epsilon)$, as required by Lemma 4.1, Algorithm 2 only requires a $1/N^2 = O(\epsilon^2)$ approximation to the Chebyshev polynomial moments, which is rather coarse in that it depends polynomially, rather than exponentially, on ϵ . As we will see in the next section, such an approximation is easily obtained with $\text{poly}(1/\epsilon)$ matrix-vector multiplications with A , and for some matrices, can even be obtained with a sublinear time algorithm.

Proof. We first prove that q is a probability distribution. Let s_N denote the ideal distribution returned by Algorithm 1 if exact Chebyshev moments were used. I.e.,

$$s_N = w \cdot \sum_{k=0}^N \frac{\hat{b}_N[k]}{\hat{b}_N[0]} \cdot \tau_k \cdot \bar{T}_k$$

where $\tau_k = \frac{1}{n} \text{tr}(\bar{T}_k(A)) = \langle s, \bar{T}_k \rangle$. Note that for any density s , $\tau_0 = \langle s, \bar{T}_0 \rangle = 1/\sqrt{\pi}$. Let $\Delta_k = \tilde{\tau}_k - \tau_k$. We have $\tilde{s}_N(x) = s_N + \sum_{k=1}^N \Delta_k \frac{\hat{b}_N[k]}{\hat{b}_N[0]} w(x) \bar{T}_k(x)$. Define functions $\eta = s_N/w$ and $\tilde{\eta} = \tilde{s}/w$. It follows that for any $x \in [-1, 1]$,

$$|\tilde{\eta}(x) - \eta(x)| = \left| \sum_{k=1}^N \frac{\hat{b}_N[k]}{\hat{b}_N[0]} \Delta_k \bar{T}_k(x) \right| \leq \frac{\sqrt{2}}{N\sqrt{\pi}}. \quad (7)$$

The last inequality uses that $0 \leq \hat{b}_N[k]/\hat{b}_N[0] \leq 1$ and for $x \in [-1, 1]$, $\bar{T}_k(x) \leq \sqrt{2/\pi}$ for $k \geq 1$. Since η is a non-negative function, from (7) we can conclude that the function $\tilde{\eta} + \frac{\sqrt{2}}{N\sqrt{\pi}}$ is non-negative, and thus $w \cdot (\tilde{\eta} + \frac{\sqrt{2}}{N\sqrt{\pi}}) = \tilde{s}_N + w \frac{\sqrt{2}}{N\sqrt{\pi}}$ is also non-negative. The density of this function is $\int_{-1}^1 \tilde{s}_N(x) dx + \frac{\sqrt{2}}{N\sqrt{\pi}} \int_{-1}^1 w(x) dx = 1 + \frac{\sqrt{2\pi}}{N}$, so dividing by $1 + \frac{\sqrt{2\pi}}{N}$ gives a probability density.

Next we prove the approximation guarantee. By Lemma 3.4 we know that $W_1(s, s_N) \leq \epsilon$, so if we can show that $W_1(s_N, q) \leq \epsilon$, then by triangle inequality we will have shown that $W_1(s, q) \leq W_1(s, s_N) + W_1(s_N, q) \leq 2\epsilon$.

To bound $W_1(s_N, q)$, we need to show that $\langle f, s_N - q \rangle \leq \epsilon$ for any 1-Lipschitz function $f \in \mathcal{F}([-1, 1], \mathbb{R})$. Without loss of generality, we can assume that $\int_{-1}^1 f(x) dx = 0$, as the 1-Lipschitz function $f' = f - \int_{-1}^1 f(x) dx$ satisfies $\langle f, s_N - q \rangle = \langle f', s_N - q \rangle$ (since s_N and q are both probability densities). If $\int_{-1}^1 f(x) dx = 0$, $f(x)$ must be zero for some $x \in [-1, 1]$, and since it is also 1-Lipschitz we can in turn bound $\|f\|_\infty \leq 1$.³ We can then bound the inner product:

$$\begin{aligned} \langle f, s_N - q \rangle &\leq \|f(\tilde{s}_N - q)\|_1 \leq \|f\|_\infty \|\tilde{s}_N(x) - q(x)\|_1 \leq \left\| w \cdot \left(\eta - \frac{\tilde{\eta} + \frac{\sqrt{2}}{N\sqrt{\pi}}}{1 + \frac{\sqrt{2\pi}}{N}} \right) \right\|_1 \\ &\leq \underbrace{\left\| w \cdot \left(\eta - \tilde{\eta} - \frac{\sqrt{2}}{N\sqrt{\pi}} \right) \right\|_1}_{z_1} + \underbrace{\left\| \frac{\sqrt{2\pi}}{N} \cdot w \cdot \left(\tilde{\eta} + \frac{\sqrt{2}}{N\sqrt{\pi}} \right) \right\|_1}_{z_2} \end{aligned}$$

The last inequality uses the fact that $1 - \frac{1}{1+\gamma} \leq \gamma$ for $0 \leq \gamma \leq 1$, which we apply with $\gamma = \frac{\sqrt{2\pi}}{N}$. Using the fact that $\|w\|_1 = \int_{-1}^1 \frac{1}{\sqrt{1-x^2}} dx = \pi$ and the bound on $\|\eta - \tilde{\eta}\|_\infty$ from (7), we have

$$z_1 \leq \|w\|_1 \cdot \left\| \eta - \tilde{\eta} - \frac{\sqrt{2}}{N\sqrt{\pi}} \right\|_\infty \leq \frac{2\pi\sqrt{2}}{N\sqrt{\pi}}.$$

Examining z_2 , recall that we showed earlier that $w(\tilde{\eta} + \frac{\sqrt{2}}{N\sqrt{\pi}}) = \tilde{s}_N + \frac{\sqrt{2}}{N\sqrt{\pi}} w$ has ℓ_1 norm $1 + \frac{\sqrt{2\pi}}{N}$. So we have $z_2 \leq \frac{\sqrt{2\pi}}{N} (1 + \frac{\sqrt{2\pi}}{N}) \leq \frac{2\sqrt{2\pi}}{N}$ for all $N \geq 1$.

Compiling the bounds on z_1 and z_2 , we have that for all 1-Lipschitz f , $\langle f, s_N - q \rangle \leq \frac{4\sqrt{2\pi}}{N} \leq \frac{11}{N}$, and thus $W_1(\tilde{s}_N, q) \leq \frac{11}{N}$. For $N \geq \frac{18}{\epsilon}$ we conclude that $W_1(\tilde{s}_N, q) \leq \epsilon$. Applying triangle quality as discussed above completes the proof. $\square$

³Let z maximize $f(x)$. Since f is 1-Lipschitz we have $f(z) \leq |x - z| - f(x)$ for all x . Integrating both sides from -1 to 1 , we have $2f(z) \leq (z^2 + 1) - 0 \leq 2$. So, $f(z) \leq 1$.

5 Efficient Chebyshev Moment Approximation

With Lemma 4.1 in place, we are ready to prove our main results. To do so, we need to show how to efficiently approximate the first N Chebyshev moments of a matrix A 's spectral density s , as required by Algorithm 2. Recall that the k^{th} normalized Chebyshev moment of s is equal to $\langle s, \bar{T}_k \rangle = \frac{1}{n} \text{tr}(\bar{T}_k(A))$. We will prove that this trace can be approximated using Hutchinson's stochastic trace estimator, implemented with either exact or approximate matrix-vector multiplications with A .

This estimator requires repeatedly computing $\bar{T}_k(A)g$ for a random vector g , which is done using the standard three-term (forward) recurrence for the Chebyshev polynomials and requires a total of k matrix-vector multiplications with A . We analyze the basis approach in Section 5.1, which yields Theorem 1.1. Then in Section 5.2, we argue that the approach is stable even when implemented with approximate matrix-vector multiplication, which yields Theorem 1.3.

5.1 Exact Matrix-Vector Multiplications

Hutchinson's estimator is a widely used estimator to efficiently compute accurate estimates of $\text{tr}(R)$ for any square matrix $R \in \mathbb{R}^{n \times n}$. Each instance of the estimator computes the quadratic form $g^\top R g$ for a random vector $g \in \{-1, 1\}^n$ whose entries are Rademacher random variables. This is an unbiased estimator for $\text{tr}(R)$ with variance $\leq 2\|R\|_F^2$, and its error has been analyzed in several earlier results [AT11, RA15]. We apply a standard high-probability bound from [MMM20, RV13]:

Lemma 5.1 (Lemma 2, [MMM20]).⁴ *Let $R \in \mathbb{R}^{n \times n}$, $\delta \in (0, 1/2]$, $l \in \mathbb{N}$. Let $g^{(1)}, \dots, g^{(l)} \in \{-1, 1\}^{n \times n}$ be l random vectors with i.i.d $\{-1, +1\}$ random entries. For a fixed constant C , with probability at least $1 - \delta$,*

$$\left| \text{tr}(R) - \frac{1}{l} \sum_{i=1}^l (g^{(i)})^\top R g^{(i)} \right| \leq \frac{C \log(1/\delta)}{\sqrt{l}} \|R\|_F.$$

For a polynomial $p \in \mathcal{F}([-1, 1], \mathbb{R})$ with degree k , applying Hutchinson's estimator to $R = p(A)$ requires computing $p(A)g$, which can always be done with k matrix-vector multiplies with A . If $p(x)$ admits a recursive construction, like the Chebyshev polynomials, then this recurrence can be used. Specifically, for the Chebyshev polynomials, we have:

$$\begin{aligned} T_0(A)g &= g & T_1(A)g &= Ag \\ T_k(A)g &= 2A \cdot T_{k-1}(A)g - T_{k-2}(A)g & \text{for } k \geq 2. \end{aligned} \tag{8}$$

A moment estimation algorithm based on Hutchinson's estimator is stated as Algorithm 3.

Remark. In total, Algorithm 3 requires $N \cdot \ell$ matrix multiplications with A since for each i $T_1(A)g^{(i)}, \dots, T_N(A)g^{(i)}$ can be computed using the same N steps of the (8) recurrence. It requires $O(n\ell N)$ additional runtime to compute and sum all inner products of the form $(g^{(i)})^\top T_k(A)g^{(i)}$.

Our main bound on the accuracy of Algorithm 3 follows:

⁴In [MMM20] the lemma is stated with an assumption that $\ell > O(1/\delta)$. However, it is easy to see that the same claim holds without this assumption, albeit with a quadratically worse $\log(1/\delta)$ dependence. The proof follows from same application of the Hanson-Wright inequality used in that work.

Algorithm 3 Hutchinson Moment Estimator

Input: Symmetric $A \in \mathbb{R}^{n \times n}$ with $\|A\|_2 \leq 1$, degree $N \in 4\mathbb{N}^+$, number of repetitions $\ell \in \mathbb{N}^+$.

Output: Approximation $\tilde{\tau}_k$ to moment $\frac{1}{n} \text{tr}(\bar{T}_k(A))$ for all $k \in 1, \dots, N$.

- 1: Draw $g^{(1)}, \dots, g^{(\ell)} \sim \text{Uniform}(\{-1, 1\}^n)$.
 - 2: For $k = 1, \dots, N$, $\tilde{\tau}_k \leftarrow \frac{\sqrt{2/\pi}}{\ell n} \sum_{i=1}^{\ell} (g^{(i)})^\top T_k(A) g^{(i)}$. $\triangleright$ Computed using recurrence in (8)
 - 3: Return $\tilde{\tau}_1, \dots, \tilde{\tau}_N$.
-

Lemma 5.2. *If Algorithm 3 is run with $\ell = \max\left(1, C \cdot \log^2(N/\delta)/(n\Delta^2)\right)$, where C is a fixed positive constant, then with probability $1 - \delta$ the approximate moments returned satisfy $|\tilde{\tau}_k - \frac{1}{n} \text{tr}(\bar{T}_k(A))| \leq \Delta$ for all $k = 1, \dots, N$.*

Proof. Fix $k \in \{1, \dots, N\}$. Note that $\frac{1}{n} \text{tr}(\bar{T}_k(A)) = \frac{\sqrt{2/\pi}}{n} \text{tr}(T_k(A))$. Let C be the constant from Lemma 5.1. If $\ell = \max\left(1, C^2 \cdot \log^2(N/\delta)/(n\Delta^2)\right)$, then by that lemma we have that with probability at least $1 - \delta/N$:

$$\left| \tilde{\tau}_k - \frac{\sqrt{2/\pi}}{n} \text{tr}(T_k(A)) \right| \leq \frac{1}{n} \frac{C \log(N/\delta)}{\sqrt{\ell}} \|T_k(A)\|_F \leq \frac{C \sqrt{2/\pi}}{\sqrt{n}} \sqrt{\frac{\log(N/\delta)}{\ell}} \leq \Delta.$$

The second to last inequality follows from the fact that $\|T_k(A)\|_2 \leq 1$ and thus $\|T_k(A)\|_F \leq \sqrt{n}$. Applying a union bound over all $k \in 1, \dots, N$ gives the claim. $\square$

Theorem 1.1 immediately follows as a corollary of Lemma 5.2 and Lemma 4.1.

Proof of Theorem 1.1. We implement Algorithm 2 with Algorithm 3 used as a subroutine to approximate the Chebyshev polynomial moments, which requires setting $\Delta = \frac{1}{N^2}$. By Lemma 5.2, we conclude that we need to set $\ell = \max\left(1, CN^2 \log^2(N/\delta)/n\right)$. Then, by Lemma 4.1, setting $N = O(1/\epsilon)$ ensures that Algorithm 2 returns a distribution q which is ϵ close to A 's spectral density s in Wasserstein distance. $\square$

5.2 Approximate Matrix-Vector Multiplications

Algorithm 3 assumes access to an oracle for computing exact matrix-vector multiplies with A . In this section, we show that the method continues to work well even when each term in Hutchinson's estimator, $g^\top T_k(A)g$, is computed using an approximate matrix-vector multiplication oracle for A (see Definition 1.2). As discussed in Section 1.1, the robustness of the estimator allows the kernel polynomial method to be applied in many settings where A can only be accessed implicitly. It also forms the basis of our sublinear time algorithm for computing the spectral density of a normalized graph adjacency or Laplacian matrix, which are presented in the Section 6.

To show that approximate matrix-vector multiplications suffice, we leverage well understood stability properties of the three-term forward recurrence for Chebyshev polynomials of the first kind [Cle55, MMS18]. These properties allow us to analyze the cumulative error when $T_k(A)g$ is computed via this recurrence. Specifically, we analyze the following algorithm:

Algorithm 4 Hutchinson Moment Estimator w/ Approximate Multiplications

Input: Symmetric $A \in \mathbb{R}^{n \times n}$ with $\|A\|_2 \leq 1$, degree $N \in 4\mathbb{N}^+$, number of repetitions $\ell \in \mathbb{N}^+$, ϵ_{MV} -approximate matrix vector multiplication oracle AMV for A (see Definition 1.2).

Output: Approximation $\tilde{\tau}_k$ to moment $\frac{1}{n} \text{tr}(\bar{T}_k(A))$ for all $k \in 1, \dots, N$.

```

1: for  $i = 1, \dots, \ell$  iterations do
2:   Draw  $g \sim \text{Uniform}(\{-1, 1\}^n)$ .
3:    $\tilde{v}_0 \leftarrow g, \tilde{v}_1 \leftarrow \text{AMV}(A, g, \epsilon_{\text{MV}})$ .
4:    $\tilde{\tau}_{1,i} \leftarrow g^T \tilde{v}_1$ 
5:   for  $k = 2$  to  $N$  do
6:      $\tilde{v}_k \leftarrow 2 \cdot \text{AMV}(A, \tilde{v}_{k-1}, \epsilon_{\text{MV}}) - \tilde{v}_{k-2}$ .
7:      $\tilde{\tau}_{k,i} \leftarrow g^T \tilde{v}_k$ 
8: For  $k = 1, \dots, N, \tilde{\tau}_k \leftarrow \frac{1}{\ell} \sum_{i=1}^{\ell} \tilde{\tau}_{k,i}$ .
9: Return  $\tilde{\tau}_1, \dots, \tilde{\tau}_N$ .

```

Algorithm 4 assumes access to an approximate matrix-vector multiplication oracle for A with error ϵ_{MV} (recall Definition 1.2). Since $\|A\|_2 \leq 1$, for any vector y , we have that:

$$\|\text{AMV}(A, y, \epsilon_{\text{MV}}) - Ay\|_2 \leq \epsilon_{\text{MV}} \|y\|_2. \quad (9)$$

The algorithm uses this oracle to apply the recurrence from (8), approximately computing each $T_k(A)g$ for $k = 1, \dots, N$, which in turn allows us to approximately compute $g^T T_k(A)g$. Note that when $\epsilon_{\text{MV}} = 0$, Algorithm 4 is exactly equivalent to Algorithm 3.

Notation. Analyzing this approach requires accounting for error accumulates across iterations. To do so, we introduce some basic notation. Let v_k denote the true value of $T_k(A)g$, and let $\tilde{v}_k$ denote our computed approximation. We initialize the recurrence with $\tilde{v}_{-1} = \vec{0}$ and $\tilde{v}_0 = v_0 = g$. For $k = 0, \dots, N-1$, let $w_k = \text{AMV}(A, \tilde{v}_k, \epsilon_{\text{MV}})$ and note that $\|w_k - A\tilde{v}_k\|_2 \leq \epsilon_{\text{MV}} \|\tilde{v}_k\|_2$. In iteration k of the recurrence, we compute $\tilde{v}_{k+1}$ by applying the recurrence:

$$\tilde{v}_{k+1} := 2w_k - \tilde{v}_{k-1}.$$

For each $i \in 0, \dots, N$ we denote:

- $\delta_k := v_k - \tilde{v}_k$, with $\delta_0 = \vec{0}$. This is the *accumulated error* up to iteration k .
- $\xi_{k+1} := A\tilde{v}_k - w_k$, with $\xi_0 = 0$. $2\xi_{k+1}$ is the *new error* introduced in iteration k due to approximate matrix-vector multiplication.

As in Clenshaw's classic work [Cle55], it can be shown that δ_k itself evolves according to a *simple recurrence*, which ultimately lets us show that it can be expressed as a summation involving Chebyshev polynomials of the *second* kind, which are easily bounded. Specifically, we have:

Fact 5.3. $\delta_1 = \xi_1$ and for $2 \leq k \leq N$, $\delta_k = 2A\delta_{k-1} - \delta_{k-2} + 2\xi_k$.

Proof. The claim for δ_1 is direct since $v_0 = \tilde{v}_0$: we have $\delta_1 = v_1 - \tilde{v}_1 = Av_0 - w_0$. For $2 \leq k \leq N$, we prove the claim by writing the difference $\delta_k = v_k - \tilde{v}_k = v_k - 2(A\tilde{v}_{k-1} + \xi_k) + \tilde{v}_{k-2}$. We can then replace $v_k = 2Av_{k-1} - v_{k-2}$ and substitute in $(v_{k-1} - \tilde{v}_{k-1}) = \delta_{k-1}$ and $(v_{k-2} - \tilde{v}_{k-2}) = \delta_{k-2}$. $\square$

The Chebyshev polynomials of the second kind are defined via the following recurrence:

Definition 5.4 (Chebyshev Polynomials of the Second Kind). For $k \in \mathbb{N}^{\geq 0}$ the k -th Chebyshev polynomial of the second kind $U_k(x)$ is given by

$$\begin{aligned} U_0(x) &= 1 & U_1(x) &= 2x \\ U_k(x) &= 2x \cdot U_{k-1}(x) - U_{k-2}(x) & \text{for } k \geq 2. \end{aligned}$$

We also define $U_{-1}(x) = 0$, which is consistent with the recurrence.

Using these polynomials, we can characterize the accumulated error δ_k in terms of the error introduced in each of the prior iterations.

Lemma 5.5. For $k = 1, \dots, N$, we have

$$\delta_k = U_{k-1}(A)\xi_1 + 2 \sum_{i=2}^k U_{k-i}(A)\xi_i. \quad (10)$$

Proof. We prove the lemma by induction on $j \leq k$. For $j = 0$, the lemma is trivial since $\delta_0 = 0$ by definition and $U_{-1}(A) = 0$. For $j = 1$, $\delta_1 = \xi_1 = U_0(A)\xi_1$. By Fact 5.3, for $2 \leq j < k$, we have:

$$\delta_j = 2\xi_j + \underbrace{2A\delta_{j-1} - \delta_{j-2}}_{z_1}. \quad (11)$$

We can apply the inductive hypothesis on z_1 and recombine terms using Definition 5.4 to get:

$$\begin{aligned} z_1 &= 2A \cdot \left(U_{j-2}(A)\xi_1 + 2 \sum_{i=2}^{j-1} U_{j-1-i}(A)\xi_i \right) - U_{j-3}(A)\xi_1 - 2 \sum_{i=2}^{j-2} U_{j-2-i}(A)\xi_i \\ &= U_{j-1}(A)\xi_1 + U_1(A) \cdot 2\xi_{j-1} + \sum_{i=2}^{j-2} (2AU_{j-1-i}(A) - U_{j-2-i}(A)) \cdot 2\xi_i \\ &= U_{j-1}(A)\xi_1 + \sum_{i=2}^{j-1} U_{j-i}(A) \cdot 2\xi_i \end{aligned}$$

Noting that plugging into (11) and noting that $2\xi_j = 2U_0(A)\xi_j$ completes the proof. $\square$

Our goal is to use Lemma 5.5 to establish that δ_k is small because each ξ_i is small. It is well known that the Chebyshev polynomials of the second kind satisfy the following bounds for any $k \in \mathbb{N}$:

$$|U_k(x)| \leq k + 1 \quad \text{for} \quad x \in [-1, 1]. \quad (12)$$

This is the upper bound we need to proceed. Specifically, we will show that each estimator using Algorithm 4, $g^\top \tilde{v}_k$, well approximates Hutchinson's estimator $g^\top T_k(A)g = g^\top v_k$.

Claim 5.6. For quantities $v_k, \tilde{v}_k$ and $0 \leq \epsilon_{MV} \leq 1/2k^2$, we have

$$\left| g^\top T_k(A)g - g^\top \tilde{v}_k \right| \leq 2\epsilon_{MV} \cdot (k+1)^2 \|g\|_2^2.$$

Proof. By the definition of δ_k , we have $|g^\top T_k(A)g - g^\top \tilde{v}_k| = |g^\top \delta_k|$. By Cauchy-Schwarz we can bound $|g^\top \delta_k| \leq \|g\|_2 \|\delta_k\|_2$. We are left to bound $\|\delta_k\|_2$. Applying Lemma 5.5 and triangle inequality, we have

$$\|\delta_k\|_2 \leq \|U_{k-1}(A)\|_2 \|\xi_1\|_2 + \sum_{i=2}^k 2\|U_{k-i}(A)\|_2 \|\xi_i\|_2$$

Then applying (12) and the fact that $\|A\|_2 \leq 1$, we have $\|U_{k-i}(A)\|_2 \leq (k-i+1)$. Hence,

$$\|\delta_k\|_2 \leq k\|\xi_1\|_2 + \sum_{i=2}^k 2(k-i+1)\|\xi_i\|_2 \leq \sum_{i=1}^k 2(k-i+1)\|\xi_i\|_2.$$

Using that $\xi_i \leq \epsilon_{\text{MV}} \|\tilde{v}_{i-1}\|_2$, and that $\|T_i(A)\|_2 \leq 1$ for all i and thus $\|v_i\|_2 \leq \|g\|_2$, we have:

$$\begin{aligned} \|\delta_k\|_2 &\leq \sum_{i=1}^k 2(k-i+1) \epsilon_{\text{MV}} \|\tilde{v}_{i-1}\|_2 \leq 2\epsilon_{\text{MV}} \sum_{i=1}^k (k-i+1) (\|v_{i-1}\|_2 + \|\delta_{i-1}\|_2) \\ &\leq \epsilon_{\text{MV}} k(k+1) \left(\|g\|_2 + \max_{i \leq k} \|\delta_i\|_2 \right). \end{aligned}$$

Inducting on δ_j for $j \leq k$ gives us $\|\delta_k\|_2 \leq 2\epsilon_{\text{MV}}(k+1)^2 \|g\|_2$, which completes the proof. $\square$

Lemma 5.7. *If Algorithm 4 is run with $\ell = \max\left(1, C \cdot \log^2(N/\delta)/(n\Delta^2)\right)$ and $\epsilon_{\text{MV}} = \Delta/4N^2$, where C is a fixed positive constant, then with probability $1 - \delta$ the approximate moments returned satisfy $|\tilde{\tau}_k - \frac{1}{n} \text{tr}(\bar{T}_k(A))| \leq \Delta$ for all $k = 1, \dots, N$.*

Proof. Fix $k \in \{1, \dots, N\}$. Let $g^{(1)}, \dots, g^{(\ell)}$ be the random vectors drawn in the outer for-loop of Algorithm 4. Let $\{\tilde{v}_k^{(i)}\}_{i \in [\ell]}$ be the ℓ vectors computed by the inner for-loop and let $\{\delta_k^{(i)} := \tilde{v}_k^{(i)} - T_k(A)g^{(i)}\}_{i \in [\ell]}$ be the ℓ error vectors. Recalling that $\frac{1}{n} \text{tr}(\bar{T}_k(A)) = \frac{\sqrt{2/\pi}}{n} \text{tr}(T_k(A))$, we have:

$$\left| \tilde{\tau}_k - \frac{\sqrt{2/\pi}}{n} \text{tr}(T_k(A)) \right| \leq \frac{\sqrt{2/\pi}}{n\ell} \sum_{i=1}^{\ell} |(g^{(i)})^\top \delta_k^{(i)}| + \left| \frac{\sqrt{2/\pi}}{n\ell} \sum_{i=1}^{\ell} (g^{(i)})^\top T_k(A)g^{(i)} - \frac{1}{n} \text{tr}(T_k(A)) \right|$$

Applying Claim 5.6 and Lemma 5.1, with probability at least $1 - \delta/N$, we thus have

$$|\tilde{\tau}_k - \frac{1}{n} \text{tr}(\bar{T}_k(A))| \leq 2(k+1)^2 \epsilon_{\text{MV}} \cdot \frac{\sqrt{2/\pi}}{n\ell} \sum_{i=1}^{\ell} \|g^{(i)}\|_2^2 + \Delta/2 \leq \Delta/2 + \Delta/2.$$

The last inequality follows from the fact that $\|g^{(i)}\|_2^2 = n$ for all $i \in [\ell]$, and the choice of $\epsilon_{\text{MV}} = \Delta/4N^2$. Applying a union bound over all $k = 1, \dots, N$ gives the claim. $\square$

Theorem 1.3 immediately follows.

Proof of Theorem 1.3. We implement Algorithm 2 with Algorithm 4 used as a subroutine to approximate the Chebyshev polynomial moments, which requires setting $\Delta = \frac{1}{N^2}$. By Lemma 5.7, we conclude that we need to set $\ell = \max\left(1, CN^2 \log^2(N/\delta)/n\right)$ and $\epsilon_{\text{MV}} = 1/4N^4$. Then, by Lemma 4.1, setting $N = O(1/\epsilon)$ ensures that Algorithm 2 returns a distribution q which is ϵ close to A 's spectral density s in Wasserstein distance. $\square$

6 Sublinear Time Methods for Graphs

An interesting application of Theorem 1.3 is that it allows for the kernel polynomial method of Algorithm 2 to be combined with any randomized algorithm for approximating matrix-vector multiplications with A . In this section we show that, for common matrices derived from undirected graphs, such an algorithm can actually be implemented in *sublinear time*, leading to a sublinear time spectral density estimation (SDE) algorithm for computing graph spectra from these matrices.

Computational Model. Let $A \in \mathbb{R}^{n \times n}$ be the adjacency matrix for an unweighted, n -vertex graph $G = (V, E)$ and let $\bar{A} = D^{-1/2}AD^{-1/2}$ be the symmetric normalized adjacency matrix, where D is an $n \times n$ diagonal matrix containing the degree of each node in V . For a node i , let $\mathcal{N}(i) = \{j : (j, i) \in E\}$ denote the set of i 's neighboring nodes. We assume a computational model where, in constant time, we can 1) uniformly sample a random vertex, 2) uniformly sample a random neighbor of any vertex i , and 3) access the degree d_i of any vertex i . We also assume that we can compute all neighbors of vertex i in $O(d_i)$ time.⁵

Using this model for accessing the adjacency matrix, we show that, for any $\epsilon_{MV} \in (0, 1)$ and failure probability $\delta \in (0, 1)$, an ϵ_{MV} -approximate matrix-vector multiplication oracle for $\bar{A}$ can be implemented in $O(n \epsilon_{MV}^{-2} \log(1/\delta))$ time. Via Theorem 1.3, this immediately yields an algorithm for computing an SDE that is ϵ close in Wasserstein-1 distance to $\bar{A}$'s spectral density in roughly $O(n/\epsilon^9)$ time for sufficiently large n , and at most $O(n/\epsilon^{13})$ time, for fixed δ . Our main result is stated as Theorem 1.4 in Section 1.1.

The same algorithm can be used to approximate the spectral density of the normalized Laplacian of G by a simple shift and scaling. Specifically, $\bar{A}$ can be obtained from the normalized Laplacian $\bar{L}$ via $\bar{A} = I - \bar{L}$, and the spectral density of $\bar{L}$, $s_{\bar{L}}(x)$ satisfies $s_{\bar{L}}(1 - x) = s_{\bar{A}}(x)$, where $s_{\bar{A}}$ is the spectral density of $\bar{A}$. So if we obtain an ϵ -approximate SDE q for $\bar{A}$ by Theorem 1.4, then the function p satisfying $p(1 - x) = q(x)$ is an ϵ -approximate SDE for $s_{\bar{L}}$. We thus have:

Corollary 6.1. *Given the normalized adjacency matrix of G , there exists an algorithm that takes $O\left(n \text{poly}\left(\frac{\log(1/\delta)}{\epsilon}\right)\right)$ expected time and outputs a density function q that is ϵ close to the spectral density of the normalized Laplacian of G with probability at least $1 - \delta$.*

6.1 Approximate Matrix-Vector Multiplication for Adjacency Matrices

We implement an approximate matrix-vector multiplication oracle for $\bar{A}$ in Algorithm 5, which is inspired by a randomized matrix-multiplication method of [DKM06]. Throughout this section, let $\bar{A}^i$ denote the i^{th} column of $\bar{A}$. Given a sampling budget $t \in \mathbb{N}$, the algorithm samples t indices from $1, \dots, n$ independently and with replacement – i.e., the same index might be sample multiple times. For each index it samples, the algorithm decides to accept or reject the column corresponding to that index with some probability. To approximate $\bar{A}y$, the algorithm outputs the multiplication of the accepted columns, rescaled appropriately, with the corresponding elements of y .

The following lemma bounds the expected squared error of Algorithm 5's:

⁵A standard adjacency list representation of the graph would allow us to perform these operations in the stated running time, although weaker access models would also suffice – e.g., random crawl access to a network [KLS11]

Algorithm 5 AMV Multiplication Oracle for Normalized Adjacency Matrices

Input: Normalized adjacency matrix $\bar{A} \in \mathbb{R}^{n \times n}$, degrees $[d_1, \dots, d_n]$, $y \in \mathbb{R}^n$, and parameter $t \in \mathbb{N}$.

Output: A vector $z \in \mathbb{R}^n$ that approximates $\bar{A}y$.

```
1: Initialize  $z \leftarrow \vec{0}$ .
2: for  $t$  iterations do
3:   Sample a node  $j$  uniformly at random from  $\{1, \dots, n\}$ .
4:   Sample a neighbor  $i \in \mathcal{N}(j)$  uniformly at random.
5:   Sample  $x$  uniformly at random from  $[0, 1]$ .
6:   if  $x \leq \frac{1}{d_i}$  then
7:      $w \leftarrow \frac{1}{p_i} \cdot y_i \bar{A}^i$  where  $p_i = \frac{1}{nd_i} \sum_{j \in \mathcal{N}(i)} \frac{1}{d_j}$ .
8:   else
9:      $w \leftarrow \vec{0}$ .
10:   $z \leftarrow z + w$ .
11: return  $\frac{1}{t} z$ 
```

Lemma 6.2. Let $z \in \mathbb{R}^n$ be the output of Algorithm 5 with sampling budget t . We have:

$$\mathbf{E}[\|\bar{A}y - z\|_2^2] = \frac{n}{t} \|y\|_2^2 - \frac{1}{t} \|\bar{A}y\|_2^2$$

Proof. Let b denote $b = \bar{A}y$. Consider a single iteration of the main loop in Algorithm 5, which generates a vector w that is added to z . Let X_i be an indicator random variable that is 1 if w is set to a scaling of $\bar{A}^i$ on that iteration, and 0 otherwise. $X_i = 1$ if and only if 1) a neighbor of i is sampled at Line 3 of the algorithm, 2) i is sampled at Line 4 of the algorithm, and 3) the uniform random variable x satisfies $x < 1/d_i$. So, we see that $\Pr[X_i = 1]$ is exactly equal to $p_i = \frac{1}{nd_i} \sum_{j \in \mathcal{N}(i)} \frac{1}{d_j}$. It follows that, by the time we reach Line 11, w is an unbiased estimator for b . I.e., $\mathbf{E}[w] = b$. Of course, this also implies that $\mathbf{E}[z] = b$.

Our goal is to show that $\mathbf{E}[\|b - z\|_2^2] = \frac{n}{t} \|y\|_2^2 - \frac{1}{t} \|b\|_2^2$. Since the random vector $b - z$ has mean zero and is the average of t i.i.d. copies of the mean zero random vector $b - w$, it suffices that show:

$$\mathbf{E}[\|b - w\|_2^2] = n \|y\|_2^2 - \|b\|_2^2. \quad (13)$$

By linearity of expectation and the fact that $\mathbf{E}[w] = b$, we have

$$\mathbf{E}[\|b - w\|_2^2] = \|b\|_2^2 + \mathbf{E}[\|w\|_2^2] - 2\langle \mathbf{E}[w], b \rangle = \mathbf{E}[\|w\|_2^2] - \|b\|_2^2.$$

So to prove (13), we need to show that $\mathbf{E}[\|w\|_2^2] = n \|y\|_2^2$. We expand w in terms of the indicator random variables $X_1, \dots, X_n$. Notice that since we only sample one column in each iteration, the random variable $X_i X_j = 0$ for all $i \neq j$. Thus, we have:

$$\begin{aligned} \mathbf{E}[\|w\|_2^2] &= \sum_{k=1}^n \mathbf{E} \left[\sum_{i,j \in [n]} \frac{X_i X_j}{p_i p_j} (\bar{A}^i y_i)_k (\bar{A}^j y_j)_k \right] = \sum_{k=1}^n \mathbf{E} \left[\sum_{i=1}^n \frac{X_i^2}{p_i^2} \cdot (\bar{A}^i y_i)_k^2 \right] \\ &= \sum_{i=1}^n \frac{1}{p_i} \cdot \|\bar{A}^i y_i\|_2^2 = \sum_{i=1}^n n y_i^2 = n \|y\|_2^2 \end{aligned}$$

In the last equalities we used the fact that $\mathbf{E}[X_i^2] = p_i$ and that, for a normalized graph adjacency matrix, $\|\bar{A}^i\|_2^2 = \sum_{j \in \mathcal{N}(i)} \frac{1}{d_i d_j} = n p_i$. This proves (13), from which we conclude the lemma. $\square$

Using Lemma 6.2, we show that there is an ϵ_{MV} -approximate matrix-vector oracle for $\bar{A}$ based on Algorithm 5 with success probability at least $1 - \delta$ that runs in $O(n \epsilon_{MV}^{-2} \log^2(\frac{1}{\delta}))$ time.

Proposition 6.3. *Let $\bar{A} \in \mathbb{R}^{n \times n}$ be the symmetric normalized adjacency matrix of an n -vertex graph G and let $\epsilon_{MV}, \delta \in (0, 1)$ be fixed constants. There is an algorithm that, given a vector $y \in \mathbb{R}^n$, and access to G as described above, takes $O(n \epsilon_{MV}^{-2} \log(\frac{1}{\delta}))$ expected time and outputs a vector $z \in \mathbb{R}^n$ such that $\|z - \bar{A}y\|_2 \leq \epsilon_{MV} \|y\|_2$ with probability at least $1 - \delta$.*

Proof. By Lemma 6.2, we have that $\mathbf{E}[\|\bar{A}y - z\|_2^2] \leq \frac{n}{t} \|y\|_2^2$. Fix $t = 48n \epsilon_{MV}^{-2}$. Then, by Lemma 6.2 and Markov's inequality, we have that when Algorithm 5 is called on $\bar{A}$ with parameter t ,

$$\Pr[\|\bar{A}y - z\|_2 > \frac{\epsilon_{MV}}{4} \|y\|_2] \leq \frac{16n \|y\|_2^2}{t \epsilon_{MV}^2 \|y\|_2^2} \leq \frac{1}{4}. \quad (14)$$

In order to improve our success probability from $3/4$ to $1 - \delta$, we use the standard trick of repeating the above process $r = c \log(\frac{1}{\delta})$ times for a constant c to be fixed later. Let $z_1, \dots, z_r \in \mathbb{R}^n$ be the output of running Algorithm 5 r times with parameter t . We can return as our estimate for $\bar{A}y$ the first z_i such that $\|z_i - z_j\|_2 \leq \frac{\epsilon_{MV}}{2} \|y\|_2$ for at least $r/2 + 1$ vectors z_j from $z_1, \dots, z_r$.

To see why this works, note that a Chernoff bound can be used to claim that with probability $> 1 - \delta$, at least $r/2 + 1$ vectors z_j from $z_1, \dots, z_r$ have that $\|z_j - \bar{A}y\|_2 \leq \frac{\epsilon_{MV}}{4} \|y\|_2$.

By a triangle inequality we have that for all such z_j and z_k ,

$$\|z_j - z_k\|_2 \leq \|z_j - \bar{A}y\|_2 + \|z_k - \bar{A}y\|_2 \leq \frac{\epsilon_{MV}}{2} \|y\|_2.$$

Thus, the z_i we picked must satisfy that $\|z_i - \bar{A}y\|_2 \leq \frac{3\epsilon_{MV}}{4} \|y\|_2$ by the triangle inequality.

All that remains is to bound the expected runtime of Algorithm 5, which we will run r separate times. To do so, note that all index sampling can be done in just $O(t)$ time, since sampling a random vertex and a random neighbor of the vertex are assumed to be $O(1)$ time operations. The costly part of the algorithm is computing the sampled column w at each iteration. In the case that $w = \vec{0}$, this cost is of course zero. However, when $w = \frac{1}{p_i} \bar{A}^i y_i$ for some i , computing the column and adding it to z takes $O(d_i)$ time, which can be large in the worst case. Nevertheless, we show that it is small in expectation. This may seem a bit surprising: while nodes with high degree are more likely to be sampled by Line 4 in Algorithm 5, they are rejected with higher probability in Line 6. Formally, let $\text{nnz}(w)$ denote the number of non-zero entries in w . We have:

$$\mathbf{E}[\text{nnz}(w)] = \sum_{i=1}^n \text{nnz}(\bar{A}^i) \cdot p_i = \sum_{i=1}^n \sum_{j \in \mathcal{N}(i)} \frac{d_i}{n \cdot d_i d_j} = \frac{1}{n} \sum_{i=1}^n \sum_{j \in \mathcal{N}(i)} \frac{1}{d_j} = 1.$$

The final equality follows from expanding the double sum: since node j has exactly d_j neighbors, $\frac{1}{d_j}$ appears exactly d_j times in the sum. So $\sum_{i=1}^n \sum_{j \in \mathcal{N}(i)} \frac{1}{d_j} = n$.

We run Algorithm 5 with $t = O(n/\epsilon_{MV}^2)$ iterations, so it follows that the expected total sparsity of all w 's constructed equals $O(n/\epsilon_{MV}^2)$, which dominates the expected running time of our method. $\square$

Proof of Theorem 1.4. The accuracy and running time claim follows from combining the ϵ_{MV} -approximate vector multiplication oracle described in Proposition 6.3 with Algorithm 2, which is analyzed in Theorem 1.3. $\square$

As discussed in the introduction, Cohen et al. [CKSV18] prove a result which matches the guarantee of Theorem 1.4, but with runtime of $2^{O(1/\epsilon)}$ – i.e., with *no dependence* on n . In comparison, our result depends linearly on n , but only polynomially on $1/\epsilon$. In either case, the result is quite surprising, as the runtime is *sublinear* in the input size: A could have up to $O(n^2)$ non-zero entries.

7 Experiments

We support our theoretical results by implementing the Jackson damped kernel polynomial method (Algorithm 2) for spectral density estimation. This method is already widely used and confirmed to work well empirically when implemented with exact matrix-vector multiplies (see e.g., [WWAF06]). So, we focus on experiments that evaluate its performance when implemented with approximate matrix-vector multiplies. Specifically, we test our sublinear time randomized method for multiplication by normalized graph adjacency matrices, presented in Section 6. We construct three graphs with substantially different spectra to illustrate the robustness of the method:

- **cliquePlusMatching** is a graph with 1000 vertices containing a clique of size 500 and a perfect matching of size 500. Its normalized adjacency matrix has 251 eigenvalues at 1, 250 at -1 , and the remaining 499 very close to 0 at $-O(1/n)$.
- **hairyClique** is a graph with 1000 vertices containing a clique of size 500, with each clique vertex connected to an additional degree-1 pendant vertex. Its normalized adjacency matrix has single eigenvalues of 1 and 0, and all other eigenvalues clustered very close to 0, with half at $-O(1/\sqrt{n})$ and half at $+O(1/\sqrt{n})$.
- **hypercube** is a 16384 vertex boolean hypercube graph on 14 bit strings.⁶ Its normalized adjacency matrix has eigenvalues at $-1, -\frac{6}{7}, -\frac{5}{7}, \dots, 0, \dots, \frac{6}{7}, 1$. The multiplicity of the 0 eigenvalue is largest, with eigenvalues closer to -1 and 1 having lower multiplicity.

For each graph, we compute the normalized Chebyshev moments $\tau_1, \dots, \tau_N$ of the normalized adjacency matrix using three methods – (i) exactly based on a full eigendecomposition of the matrix, (ii) using Hutchinson’s moment estimator as in Algorithm 3, and, (iii) combining the approximate matrix-vector multiplication algorithm of Algorithm 5 with Hutchinson’s moment estimator, as in Algorithm 4. For both (ii) and (iii) just $\ell = 2$ random vectors suffice for the implementation of Hutchinson’s, which is inline with our theory: Lemma’s 5.2 and 5.7 suggest that the number of random vectors needed decreases linearly with the matrix dimension n . For **cliquePlusMatching** and **hairyClique** we approximate the first $N = 40$ Chebyshev moments and for **hypercube** we approximate the first $N = 80$. These moments are then passed into either Algorithm 1 (if they are computed exactly) or Algorithm 2 (if they are approximate). In either case, the method returns a closed form, continuous spectral density. Using a simply greedy algorithm for discretizing the density (described in Appendix A) we also obtain a list of approximate eigenvectors.

Our main results are show in Table 1, which lists the Wasserstein error of the discretized spectrum obtained from each method. “Idealized” refers to the Jackson damped KPM implemented with *exact* Chebyshev moments. This method serves as our baseline. Even with just $\ell = 2$ repetitions of Hutchinson’s the KPM method achieved less than 8% error for all graphs, and its performance was

⁶A boolean hypercube contains a vertex for each distinct b bit string, and an edge between two vertices if the corresponding strings differ on exactly 1 bit.

Graph	Wasserstein Error			Fraction of Entries Sampled in AMV Mult.
	Idealized	Hutchinson	Approximate Hutchinson	
<code>cliquePlusMatching</code>	4.2%	6.8%	5.5%	34%
<code>hairyClique</code>	4.5%	7.6%	9.9%	59%
<code>hypercube</code>	2.9%	3.7%	5.8%	67%

Table 1: Wasserstein error of the discrete spectral density obtained from each method. We also show the average fraction of non-zero matrix entries sampled for each approximate matrix-vector product computed by Algorithm 5 for the approximate Hutchinson’s moment estimator.

comparable to the idealized method. The approximate Hutchinson’s method required repeated calls to the approximate matrix-vector multiplication oracle of Algorithm 5. For a simple comparison, we increased the oversampling parameter t until the performance of the method was on-par with the method based on exact matrix vector multiplications. We then compute the average number of non-zero elements of A accessed by the method for each matrix-vector product, which reflects the runtime improvement over a full matrix-vector product. For all graphs, Algorithm 5 accessed $< \text{nnz}(A)$ matrix entries on average, showing its potential as a sublinear time algorithm. It even showed improvement for the relatively sparse `hypercube` graph, for which the standard Hutchinson’s method already runs in $O(n \log n)$ time. While the improvements are moderate, there is a non-trivial speedup over Hutchinson’s moment estimator. Comparing Theorem’s 1.1 and Theorem 1.3, the approximate Hutchinson’s method is expected to scale linearly with the graph size n , whereas Hutchinson’s scales quadratically for dense graphs like `cliquePlusMatching` and `hairyClique`. So, the improvement would likely be more significant for larger graphs.

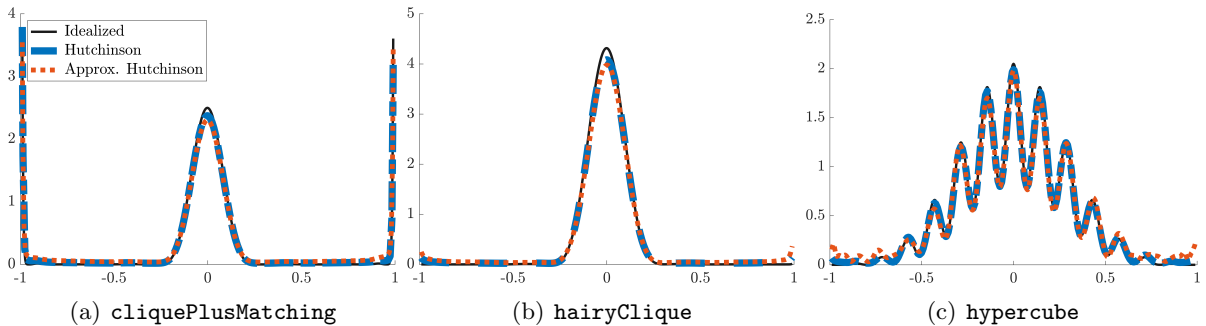


Figure 3: Spectral density estimate returned by Jackson damped kernel polynomial method using exact moments, Hutchinson’s moment estimator, and our approximate Hutchinson’s moment estimator (with $l = 2$ random vectors). The KPM was run with degree $N = 40$ for `cliquePlusMatching` and `hairyClique`, and degree $N = 80$ for `hypercube`. The Hutchinson methods closely match the baseline idealized method.

The numerical results of Table 1 can be confirmed visually. In Figure 3 we show the approximate spectral density returned by each method, and in Figure 4 we plot histograms of the approximate eigenvalues obtained after discretization. Both the Hutchinson’s method and method with approximate matrix-vector multiplications closely match the idealized KPM method.

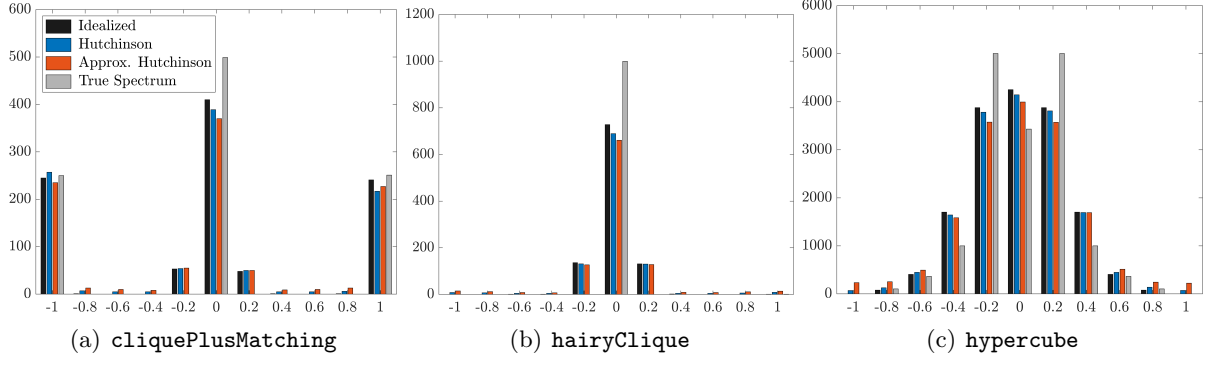


Figure 4: Histogram of eigenvalues obtained from discretizing the approximate spectral densities in Figure 3. For comparison, a histogram of the true normalized adjacency matrix eigenvalues is also shown. The interval is partitioned into 25 equally sized bins. Again the Hutchinson methods closely match the baseline idealized method.

Finally, to better understand the above results, in Figures 5 and 6 we plot both the approximate moments computed by each of the methods, as well as the moments scaled by their respective coefficient $\frac{\hat{b}_N[k]}{\hat{b}_N[0]}$ in the Jackson damped kernel polynomial method. Due to error accumulation from approximate matrix-vector multiplies, the approximate Hutchinson's moment estimator is somewhat less accurate for higher moments. However, since the Jackson damping coefficients approach 0 for the larger moments, the impact of this error is significantly less than error in computing the lower moments, so it does not affect the quality of the density returned.

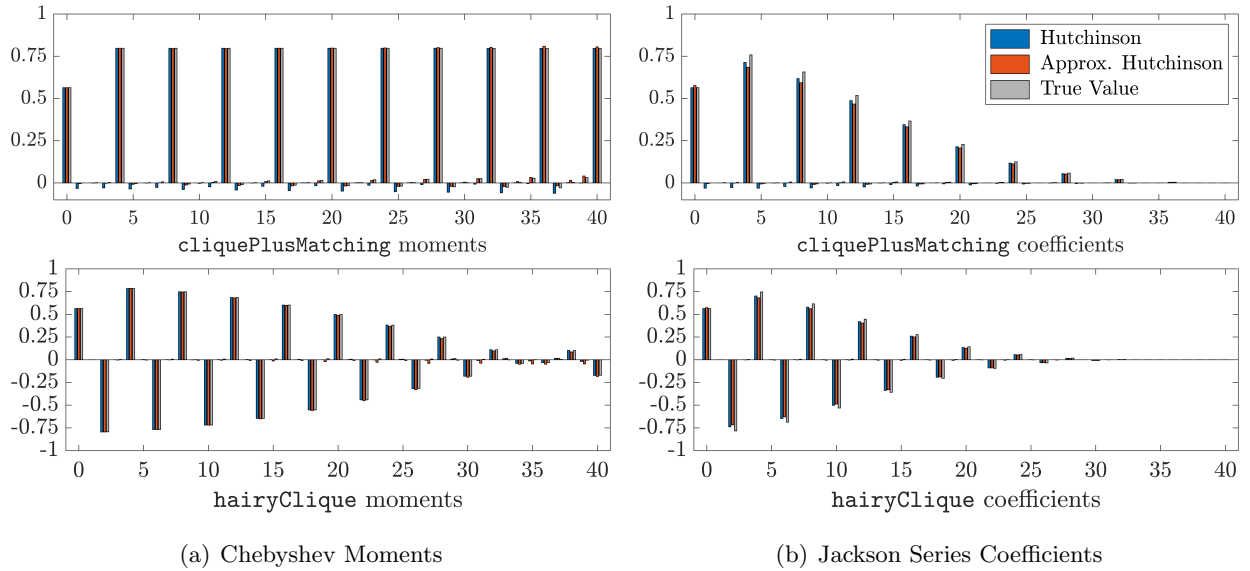


Figure 5: The normalized Chebyshev moments $\{\tau_k\}_{k \in [N]}$ for $N = 40$ using Hutchinson's and approximate Hutchinson's moment estimator and the corresponding coefficients $\{\tau_k \cdot \hat{b}_N[k] / \hat{b}_N[0]\}_{k \in [N]}$ used in the Jackson damped kernel polynomial method.

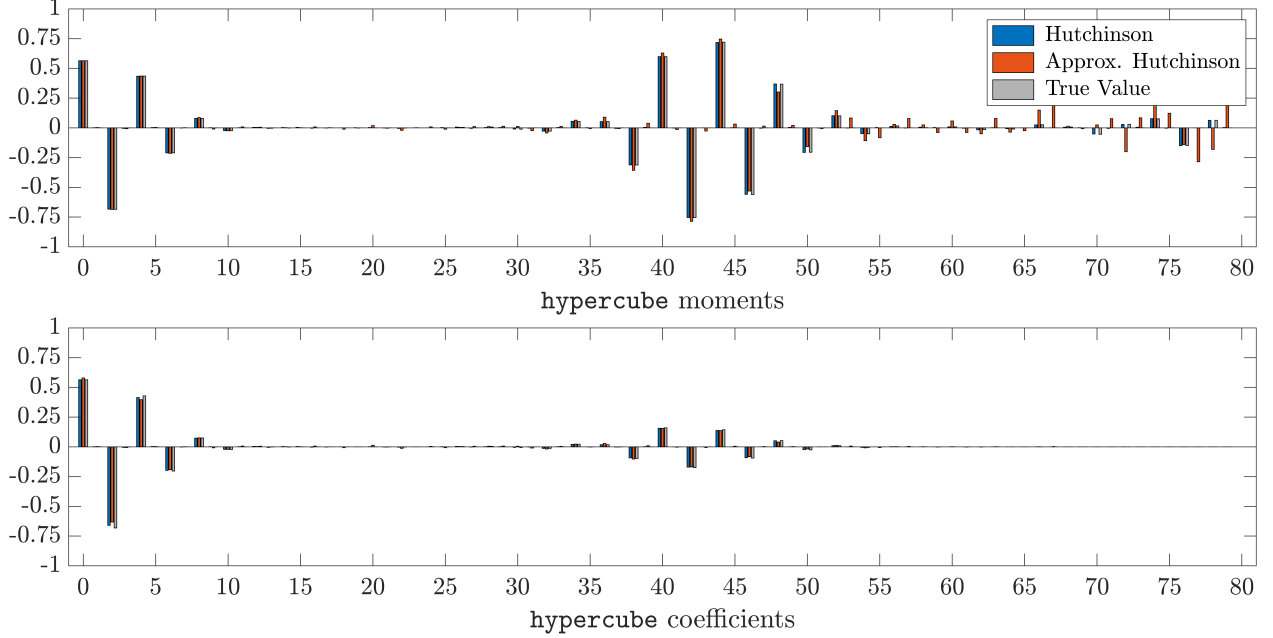


Figure 6: The normalized Chebyshev moments $\{\tau_k\}_{k \in [N]}$ for $N = 80$ using Hutchinson’s and approximate Hutchinson’s moment estimator and the corresponding coefficients $\{\tau_k \cdot \hat{b}_N[k] / \hat{b}_N[0]\}_{k \in [N]}$ used in the Jackson damped kernel polynomial method.

8 Acknowledgements

We thank Cameron Musco and Raphael Meyer for helpful discussions and technical suggestions about this work.

References

- [AKS17] Jared L. Aurentz, Vassilis Kalantzis, and Yousef Saad. Cucheb: A GPU implementation of the filtered Lanczos procedure. *Computer Physics Communications*, 220:332 – 340, 2017.
- [AT11] Haim Avron and Sivan Toledo. Randomized algorithms for estimating the trace of an implicit symmetric positive semi-definite matrix. *J. ACM*, 58(2), 2011.
- [BHSW20] Mark Braverman, Elad Hazan, Max Simchowitz, and Blake Woodworth. The gradient complexity of linear regression. In *Proceedings of the 33rd Annual Conference on Computational Learning Theory (COLT)*, volume 125, pages 627–647, 2020.
- [BVKS19] Jess Banks, Jorge Vargas, Archit Kulkarni, and Nikhil Srivastava. Pseudospectral shattering, the sign function, and diagonalization in nearly matrix multiplication time. In *Proceedings of the 61st Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, 2019.
- [CKSV18] David Cohen-Steiner, Weihao Kong, Christian Sohler, and Gregory Valiant. Approximating the spectrum of a graph. In *Proceedings of the 24th ACM SIGKDD*

- International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 1263–1271, 2018.
- [Cle55] C. W. Clenshaw. A note on the summation of chebyshev series. *Mathematics of Computation*, 9(51):118, 1955.
 - [DBB19] Kun Dong, Austin R Benson, and David Bindel. Network density of states. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 1152–1161, 2019.
 - [DKM06] Petros Drineas, Ravi Kannan, and Michael W Mahoney. Fast Monte Carlo algorithms for matrices I: Approximating matrix multiplication. *SIAM Journal on Computing*, 36(1):132–157, 2006.
 - [DM21] Prathamesh Dharangutte and Christopher Musco. Dynamic trace estimation. *Preprint*, 2021.
 - [GKX19] Behrooz Ghorbani, Shankar Krishnan, and Ying Xiao. An investigation into neural net optimization via hessian eigenvalue density. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97, pages 2232–2241, 2019.
 - [Hut90] Michael F. Hutchinson. A stochastic estimator of the trace of the influence matrix for Laplacian smoothing splines. *Communications in Statistics-Simulation and Computation*, 19(2):433–450, 1990.
 - [Jac12] Dunham Jackson. On approximation by trigonometric sums and polynomials. *Transactions of the American Mathematical society*, 13(4):491–515, 1912.
 - [Jac30] Dunham Jackson. *The Theory of Approximation*, volume 11 of *Colloquium Publications*. American Mathematical Society, 1930.
 - [KLS11] Liran Katzir, Edo Liberty, and Oren Somekh. Estimating sizes of social networks via biased sampling. In *Proceedings of the 20th International World Wide Web Conference (WWW)*, pages 597–606, 2011.
 - [KR57] Leonid Vital’evich Kantorovich and Gennadii Shlemovich Rubinshtein. On a functional space and certain extremum problems. In *Doklady Akademii Nauk*, volume 115, pages 1058–1061. Russian Academy of Sciences, 1957.
 - [KV17] Weihao Kong and Gregory Valiant. Spectrum estimation from samples. *Ann. Statist.*, 45(5):2218–2247, 10 2017.
 - [Lor66] George G. Lorentz. *Approximation of Functions*. American Mathematical Society, second edition, 1966.
 - [LSY16] Lin Lin, Yousef Saad, and Chao Yang. Approximating spectral densities of large matrices. *SIAM Review*, 58(1):34–65, 2016.
 - [LXES19] Ruipeng Li, Yuanzhe Xi, Lucas Erlandson, and Yousef Saad. The eigenvalues slicing library (EVSL): Algorithms, implementation, and software. *SIAM Journal on Scientific Computing*, 41(4):C393–C415, 2019.

- [MM19] Michael Mahoney and Charles Martin. Traditional and heavy tailed self regularization in neural network models. In Kamalika Chaudhuri and Ruslan Salakhutdinov, editors, *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97, pages 4284–4293, 2019.
- [MMMWW20] Raphael A. Meyer, Cameron Musco, Christopher Musco, and David Woodruff. Hutch++: Optimal stochastic trace estimation. *Proceedings of the 4th Symposium on Simplicity in Algorithms (SOSA)*, 2020.
- [MMS18] Cameron Musco, Christopher Musco, and Aaron Sidford. Stability of the Lanczos method for matrix function approximation. In *Proceedings of the 29th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 1605–1624, 2018.
- [Par98] Beresford N. Parlett. *The symmetric eigenvalue problem*. SIAM, 1998.
- [Pea94] Barak A. Pearlmutter. Fast exact multiplication by the hessian. *Neural computation*, 6(1):147–160, 1994.
- [PSG18] Jeffrey Pennington, Samuel Schoenholz, and Surya Ganguli. The emergence of spectral universality in deep networks. In *Proceedings of the 21st International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 1924–1932, 2018.
- [RA15] Farbod Roosta-Khorasani and Uri M. Ascher. Improved bounds on sample size for implicit matrix trace estimators. *Foundations of Computational Mathematics*, 15(5):1187–1212, 2015.
- [RV13] Mark Rudelson and Roman Vershynin. Hanson-Wright inequality and sub-Gaussian concentration. *Electronic Communications in Probability*, 18, 2013.
- [SEAR18] Max Simchowitz, Ahmed El Alaoui, and Benjamin Recht. Tight query complexity lower bounds for pca via finite sample deformed wigner law. In *Proceedings of the 50th Annual ACM Symposium on Theory of Computing (STOC)*, pages 1249–1259, 2018.
- [Ski89] John Skilling. *The Eigenvalues of Mega-dimensional Matrices*, pages 455–466. Springer Netherlands, 1989.
- [SR94] R.N. Silver and H. Roder. Densities of states of mega-dimensional hamiltonian matrices. *International Journal of Modern Physics C*, 5(4):735–753, 1994.
- [SWYZ19] Xiaoming Sun, David P. Woodruff, Guang Yang, and Jialin Zhang. Querying a matrix through matrix-vector products. In *Proceedings of the 46th International Colloquium on Automata, Languages and Programming (ICALP)*, volume 132, pages 94:1–94:16, 2019.
- [Wan94] Lin-Wang Wang. Calculating the density of states and optical-absorption spectra of large quantum systems by the plane-wave moments method. *Phys. Rev. B*, 49:10154–10158, 1994.
- [Woo14] David P. Woodruff. Sketching as a tool for numerical linear algebra. *Foundations and Trends in Theoretical Computer Science*, 10(1–2):1–157, 2014.

- [WWAF06] Alexander Weiße, Gerhard Wellein, Andreas Alvermann, and Holger Fehske. The kernel polynomial method. *Reviews of modern physics*, 78(1):275, 2006.
- [YGKM20] Zhewei Yao, Amir Gholami, Kurt Keutzer, and Michael W. Mahoney. Pyhessian: Neural networks through the lens of the hessian. In *IEEE BigData*, 2020.

A Approximate Eigenvalues from Spectral Density Estimate

Algorithm 1 and Algorithm 2 in the previous sections output a closed form representation of a distribution q which is close in Wasserstein-1 distance to s . In particular, the distribution output is continuous. Alternatively, we describe a simple greedy algorithm (Algorithm 6) that recovers a list of n eigenvalues $\tilde{\Lambda} = [\tilde{\lambda}_1, \dots, \tilde{\lambda}_n]$ such that $\|\Lambda - \tilde{\Lambda}\|_1 \leq 3n\epsilon$, which implies that the discrete distribution associated with $\tilde{\Lambda}$ is 3ϵ close to s in Wasserstein-1 distance. Formally:

Theorem A.1. *Let s be a spectral density and let q be a density on $[-1, 1]$ such $W_1(s, q) \leq \epsilon$ for $\epsilon \in (0, 1)$. As long as q can be integrated over any subinterval of $[-1, 1]$ (e.g., has a closed form antiderivative), there is an algorithm (Algorithm 6) that computes $1/\epsilon$ such integrals and in $O(n + 1/\epsilon)$ additional time outputs a list of n values $\tilde{\Lambda} = [\tilde{\lambda}_1, \dots, \tilde{\lambda}_n]$ such that $\|\Lambda - \tilde{\Lambda}\|_1 \leq 3n\epsilon$.*

At a high-level, Algorithm 6 computes a grid with spacing ϵ for the interval $[-1, 1]$, “snaps” the mass of the continuous density onto the nearest point in the grid, and then readjusts the resulting point masses to a distribution where every point mass is divisible by $1/n$ (and can therefore be represented by a certain number of eigenvalues). It does so by iteratively shifting fractional masses to the next point in the grid so that the mass at the current point is divisible by $1/n$.

The method requires computing the mass $\int_a^b q(x)dx$ where $-1 \leq a < b \leq 1$. For Algorithms 1 and 2, q is written as $q = w \cdot p$ where p is a degree N polynomial written as a sum of the first $N + 1$ Chebyshev polynomials. So to compute the integral $\int_a^b q(x)dx$, we just need to compute the integral $\int_a^b T_k(x)w(x)dx$ for any $k \in 0, \dots, N$. We can do so using the following closed form expression (see Appendix C for a short derivation):

Fact A.2. *For $k \in \mathbb{N}^{>0}$ and $-1 \leq a < b \leq 1$ we have that*

$$\int_a^b \frac{T_k(x)}{\sqrt{1-x^2}} dx = \frac{-\cos(k \sin^{-1} b)}{k} - \frac{-\cos(k \sin^{-1} a)}{k}$$

For $k = 0$, $T_k(x) = 1$ for all x and we have that $\int_a^b T_k(x)w(x)dx = \sin^{-1}(b) - \sin^{-1}(a)$.

Using the above fact, when $q = w \cdot p$ for a degree N polynomial p , we can compute $\int_a^b q(x)dx$ in $O(N)$ time. In our main results $N = O(1/\epsilon)$, so this cost is small.

Proof of Theorem A.1. Consider the output $\tilde{\Lambda}$ of Algorithm 6 with input q and n . Notice that $W_1(v, q) \leq \epsilon$ by the definition of v and the earthmover’s definition of the Wasserstein distance. Hence, by triangle inequality, we have that $W_1(v, s) \leq 2\epsilon$. Let $\tilde{v}$ be the vector of masses after the shifting procedure (Line 4) in the for-loop of the algorithm. Notice that $\tilde{v}$ is the distribution corresponding to having n equally weighted point-masses on the points in $\tilde{\Lambda}$. Since the procedure

Algorithm 6 Approximate Eigenvalues from Spectral Density

Input: Spectral density $q : [-1, 1] \rightarrow \mathbb{R}^+$, integer n .

Output: Vector $\tilde{\Lambda} = [\tilde{\lambda}_1, \dots, \tilde{\lambda}_n]$.

- 1: compute $\vec{v} = (v_{-1+\epsilon}, v_{-1+2\epsilon}, \dots, v_0, v_\epsilon, \dots, v_1)$ such that $v_t = \int_{t-\epsilon}^t q(x)dx$
 - 2: **for** t in $(-1 + \epsilon, -1 + 2\epsilon, \dots, 0, \epsilon, \dots, 1)$ **do**
 - 3: $r \leftarrow v_t - \lfloor v_t \rfloor_{1/n}$ $\triangleright \lfloor v_t \rfloor_{1/n}$ is the largest value $\leq v_t$ that is divisible by $\frac{1}{n}$
 - 4: $v_{t+\epsilon} \leftarrow r + v_{t+\epsilon}$
 - 5: Set $n \cdot \lfloor v_t \rfloor_{1/n}$ values in $\hat{\Lambda}$ to be t
 - 6: **return** $\hat{\Lambda}$
-

in Line 4 moves at most $1/n$ mass at most ϵ distance in each iteration, we have $W_1(v, \tilde{v}) \leq \epsilon$ by the earthmover's distance definition of the Wasserstein-1 distance. It follows then that $W_1(\tilde{v}, s) \leq 3\epsilon$. □

We note that there are other options beyond Algorithm 6 for discretizing a continuous density return by the Jackson damped kernel polynomial method – i) the optimal discretization of a continuous density on the interval $[-1, 1]$ into n equally-weighted point-masses, and ii) an algorithm by [CKSV18] that can be seen as a combination of Algorithm 6 and the optimal method.

Optimal Discretization. Given the continuous density q , consider the discrete density that results from the following procedure:

1. Initialize $t = -1$, then repeat the following steps until $t = 1$.
2. Let $t' \geq t$ be the smallest value such that $\int_t^{t'} q(x)dx = \frac{1}{n}$.
3. Place a point-mass at $\mathbf{E}_{x \sim q} [x \mid x \in [t, t']]$. I.e. a point-mass is placed in the interval $[t, t']$ at the point given by the conditional distribution of q on the interval.
4. Update $t \leftarrow t'$.

The values $\tilde{\Lambda} = \tilde{\lambda}_1, \dots, \tilde{\lambda}_n$ given by the point-masses computed by the aforementioned procedure is a optimal discretization of q into n equally-weighted point-masses on $[-1, 1]$ in terms of Wasserstein-1 distance. To see why this is the case, consider the first $1/n$ fraction of the mass of the density q , i.e. the smallest $t > -1$ such that $\int_{-1}^t q(x)dx = 1/n$. The policy minimizing the earthmover's distance to any n equally-weighted point-wise masses must “move” the mass of q on the interval $[-1, t]$ to the point-mass closest to -1 . Hence, it is sufficient to restrict our attention to the interval $[-1, t]$ when computing the smallest point-mass, i.e. the mass closest to -1 . Now that we are constrained to looking at the interval $[-1, t]$ one can check that the point-mass minimizing the earthmover's distance to q , restricted to $[-1, t]$, is the point-mass at $\mathbf{E}_{x \sim q} [x \mid x \in [-1, t]]$. The optimality of the procedure follows from making this argument inductively for all n point-masses.

We note that all steps of the procedure takes roughly $O(n)$ time, although a numerical integration technique or binary search would need to be used to find each t' to sufficiently high accuracy.

A result combining the greedy discretization in Algorithm 6 and the optimal discretization is given in [CKSV18]. They compute a fractional discretization on an ϵ -spaced grid of $[-1, 1]$, as in Algorithm

6, but then compute the eigenvalues using the conditional expectation of every $1/n$ fraction of mass based on the discrete density on the grid.

B Positive Polynomial Approximation

In this section, we introduce Jackson's powerful result from 1912 on the uniform approximation of Lipschitz continuous periodic functions by low-degree trigonometric polynomials [Jac12, Jac30]. This result will directly translate to the result for algebraic polynomials needed to analyze the kernel polynomial method. We start with basic definitions and preliminaries below.

B.1 Fourier Series Preliminaries

Definition B.1 (Fourier Series). A function f with period 2π that is integrable on the length of that period can be written via the Fourier series:

$$f(x) = \frac{\alpha_0}{2} + \sum_{k=1}^{\infty} \alpha_k \cos(kx) + \beta_k \sin(kx)$$

where

$$\alpha_k = \frac{1}{\pi} \int_{-\pi}^{\pi} f(x) \cos(kx) dx \quad \beta_k = \frac{1}{\pi} \int_{-\pi}^{\pi} f(x) \sin(kx) dx.$$

Equivalently we can write f in *exponential form* as:

$$f(x) = \sum_{k=-\infty}^{\infty} \hat{f}_k e^{ikx}$$

where $i = \sqrt{-1}$, $\hat{f}_0 = \alpha_0/2$, $\hat{f}_k = \hat{f}_{|k|}^*$ for $k < 0$, and for $k > 0$,

$$\hat{f}_k = \frac{1}{2}(\alpha_k - i\beta_k).$$

If the Fourier series of a periodic function f has $\hat{f}_k = 0$ for $k > N$ (equivalently, $\alpha_k = \beta_k = 0$ for $k > N$), we say that f is a degree N trigonometric polynomial.

In working with Fourier series, we require the two standard convolution theorems:

Claim B.2 (First Convolution Theorem). *Let f, g be integrable 2π -periodic functions with exponential form Fourier series coefficients $[\hat{f}_k]_{k=-\infty}^{\infty}$ and $[\hat{g}_k]_{k=-\infty}^{\infty}$, respectively. Let h be their convolution:*

$$h(x) = [f * g](x) = \int_{-\pi}^{\pi} f(u)g(x-u)du.$$

The exponential form Fourier series coefficients of h , $[\hat{h}_k]_{k=-\infty}^{\infty}$, satisfy:

$$\hat{h}_k = 2\pi \cdot \hat{f}_k \hat{g}_k$$

Claim B.3 (Second Convolution Theorem). *Let f, g be integrable 2π -periodic functions with exponential form Fourier series coefficients $[\hat{f}_k]_{k=-\infty}^{\infty}$ and $[\hat{g}_k]_{k=-\infty}^{\infty}$, respectively. Let h be their product:*

$$h(x) = f(x) \cdot g(x).$$

The exponential form Fourier series coefficients of h , $[\hat{h}_k]_{k=-\infty}^{\infty}$, satisfy:

$$\hat{h}_k = \sum_{j=-\infty}^{\infty} \hat{f}_j \cdot \hat{g}_{k-j}$$

In other words, the Fourier coefficients of h are the discrete convolution of those of f and g .

B.2 Jackson's Theorem for Trigonometric Polynomials

We seek a low-degree trigonometric polynomial $\tilde{f}$ that is a good *uniform* approximation to any sufficiently smooth periodic function f . I.e., we want $\|f - \tilde{f}\|_{\infty} < \epsilon$ where $\|z\|_{\infty}$ denotes $\|z\|_{\infty} = \max_x z(x)$. A natural choice for $\tilde{f}$ is the truncated Fourier series $\sum_{k=-N}^N c_k e^{ikx}$, but this does not lead to good uniform approximation in general. Instead, Jackson showed that better accuracy can be obtained with a Fourier series with *damped coefficients*, which is equivalent to the *convolution* of f with an appropriately chosen “bump” function (aka kernel), defined below:

Definition B.4 (Jackson Kernel). For any positive integer m , let b be the $2m - 2$ degree trigonometric polynomial:

$$b = \left(\frac{\sin(mx/2)}{\sin(x/2)} \right)^4 = \sum_{k=-2m+2}^{2m-2} \hat{b}_k e^{ikx},$$

which has exponential form coefficients $\hat{b}_{-2m+2}, \dots, \hat{b}_0, \dots, \hat{b}_{2m-2}$ equal to:

$$\hat{b}_{-k} = \hat{b}_k = \sum_{j=-m}^{m-k} (m - |j|) \cdot (m - |j + k|) \quad \text{for } k = 0, \dots, 2m - 2. \quad (15)$$

When m is odd it is easy to see that b is a degree $2m - 2$ trigonometric polynomial. Specifically, for odd m we have the well known Fourier series of the periodic sinc function $s(x) = \frac{\sin(mx/2)}{\sin(x/2)} = \sum_{k=-(m-1)/2}^{(m-1)/2} e^{ikx}$. We then apply the convolution theorem (Claim B.3) to $s(x) \cdot s(x)$. to see that $s^2(x) = \left(\frac{\sin(mx/2)}{\sin(x/2)} \right)^2$ is an $m - 1$ degree trigonometric polynomial with coefficients $c_{-k} = c_k = m - k$. Applying it again to $s^2(x) \cdot s^2(x)$ yields (15). For a derivation of (15) when m is even, we refer the reader to [Jac30] or [Lor66].

Jackson's main result is as follows. We include a short proof for completeness.

Theorem B.5 (Jackson [Jac12], see also [Jac30]). *Let f be a 2π -periodic, Lipschitz continuous function with Lipschitz constant λ . I.e., $|f(x) - f(y)| \leq \lambda|x - y|$ for all x, y . For integer m , let b be the bump function from Definition 15, with k^{th} Fourier coefficients $\hat{b}_k$. The function $\tilde{f}(x) = \frac{1}{2\pi\hat{b}_0} \int_{-\pi}^{\pi} b(u)f(x - u)du$ satisfies:*

$$\|\tilde{f} - f\|_{\infty} \leq 9 \frac{\lambda}{m}.$$

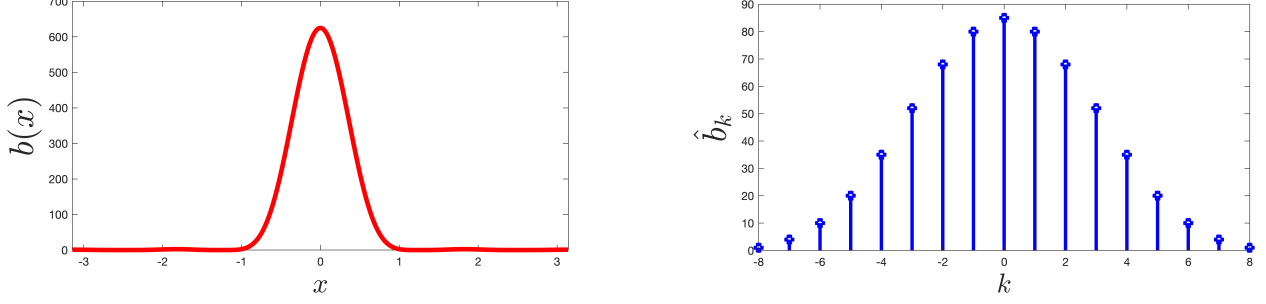


Figure 7: Jackson's bump function $b(x)$ for $m = 5$, alongside its Fourier series coefficients.

$\tilde{f}$ is a $2m-2$ degree trigonometric polynomial, and by the convolution theorem, its exponential form Fourier series coefficients are given by $\hat{\tilde{f}}_k = \frac{\hat{b}_k}{\hat{b}_0} \cdot \hat{f}_k$ for $k = -2m+2, \dots, 2m-2$.

Remark. The function $\tilde{f}$ takes the form of a *damped* truncation of f 's Fourier series: $\frac{\hat{b}_0}{\hat{b}_0} = 1$ and $\frac{\hat{b}_k}{\hat{b}_0}$ falls off towards zero as $k \rightarrow 2m-2$. After $2m-2$, the Fourier series coefficients from f are fully truncated to 0.

Proof. Recalling that $\hat{b}_0 = \frac{1}{2\pi} \int_{-\pi}^{\pi} b(x)dx$, we have that $\int_{-\pi}^{\pi} \frac{1}{2\pi\hat{b}_0} b(u)du = 1$, and thus

$$|\tilde{f}(x) - f(x)| \leq \int_{-\pi}^{\pi} \frac{1}{2\pi\hat{b}_0} b(u) \cdot |f(x) - f(x-u)| du.$$

By our Lipschitz assumption of f , we can bound $|f(x) - f(x-u)| \leq \lambda|u|$ and thus have:

$$\max_x |\tilde{f}(x) - f(x)| = \|\tilde{f} - f\|_{\infty} \leq \lambda \cdot \frac{\int_{-\pi}^{\pi} |u| b(u) du}{2\pi\hat{b}_0} = \lambda \cdot \frac{\int_0^{\pi} u b(u) du}{\int_0^{\pi} b(u) du}. \quad (16)$$

In the last equality, we use that b is symmetric about zero. We have that $2 \cdot \sin\left(\frac{u}{2}\right) \leq u \leq \pi \cdot \sin\left(\frac{u}{2}\right)$ for $x \in [0, \pi]$ and thus:

$$\int_0^{\pi} u b(u) du \leq \pi^4 \int_0^{\pi} u \frac{\sin(mu/2)^4}{u^4} du = \pi^4 m^2 \int_0^{\pi m} \frac{\sin(v/2)^4}{v^3} dv \leq \pi^4 m^2 \int_0^{\infty} \frac{\sin(v/2)^4}{v^3} dv.$$

The last integral evaluates to $\frac{\ln 2}{4}$, so overall we have $\int_0^{\pi} b(u) \cdot u du \leq \frac{\pi^4 \ln 2}{4} \cdot m^2$. Moreover we can check that:

$$\int_0^{\pi} b(u) du = \pi \cdot \left(\frac{2}{3}m^3 + \frac{1}{3}m\right) \geq \frac{2\pi}{3}m^3.$$

Plugging into (16) we have that:

$$\|\tilde{f} - f\|_{\infty} \leq 8.06 \frac{\lambda}{m}.$$

The result follows. We note that the constant above is loose: numerical results suggest the bound can be improved to $\leq \frac{\pi}{2} \frac{\lambda}{m}$. $\square$

Theorem B.5 translates to a result for *algebraic polynomials* via a standard transformation between Fourier series and Chebyshev series, which we detail below.

B.3 Jackson's Theorem for Algebraic Polynomials

Theorem B.6. *Let $f \in \mathcal{F}([-1, 1], \mathbb{R})$ be a Lipschitz continuous function on $[-1, 1]$ with Lipschitz constant λ . I.e., $|f(x) - f(y)| \leq \lambda|x - y|$ for all x, y . For integer m , let $\hat{b}_0, \dots, \hat{b}_{2m-2}$ be the coefficients from (15). Let $c_k = \langle f, w \cdot \bar{T}_k \rangle$ be the k^{th} coefficient in f 's Chebyshev polynomial expansion, where w and $\bar{T}_k$ are as defined in Section 2. The degree $(2m - 2)$ algebraic polynomial*

$$\tilde{f}(x) = \sum_{n=0}^{2m-2} \frac{\hat{b}_n}{\bar{b}_0} c_n \cdot \bar{T}_n(x)$$

satisfies $\|\tilde{f} - f\|_\infty \leq 9 \frac{\lambda}{m}$.

Proof. To translate from the trigonometric case to the algebraic setting, we will use the identity that for all k ,

$$T_k(\cos \theta) = \cos(k\theta). \quad (17)$$

Consider any function $r \in \mathcal{F}([-1, 1], \mathbb{R})$ with Chebyshev expansion coefficients $c_0, c_1, \dots$, where $c_k = \langle r, w \cdot \bar{T}_k \rangle$. Transform r into a periodic function as follows: let $g(\theta) = r(\cos \theta)$ for $\theta \in [-\pi, 0]$ and let $h(\theta) = g(-|\theta|)$ for $\theta \in [-\pi, \pi]$. The function $h(\theta)$ is periodic, and also even, so its Fourier series has all coefficients $\beta_1, \beta_2, \dots$ equal to 0. We thus have that

$$h(\theta) = \sum_{n=0}^{\infty} \alpha_n \cos(n\theta),$$

where

$$\alpha_0 = \frac{1}{2\pi} \int_{-\pi}^{\pi} h(\theta) \cos(k\theta) d\theta = \frac{1}{\pi} \int_{-\pi}^0 g(\theta) \cos(k\theta) d\theta$$

and, for $n > 0$,

$$\alpha_k = \frac{1}{\pi} \int_{-\pi}^{\pi} h(\theta) \cos(k\theta) d\theta = \frac{2}{\pi} \int_{-\pi}^0 g(\theta) \cos(k\theta) d\theta.$$

Using (17) and the fact that $\frac{d}{dx} \cos^{-1}(x) = \frac{1}{\sqrt{1-x^2}}$, we have:

$$\int_{-\pi}^0 g(\theta) \cos(k\theta) d\theta = \int_{-1}^1 r(x) T_k(x) \frac{1}{\sqrt{1-x^2}} dx.$$

We conclude that the Chebyshev coefficients of r are precisely a scaling of the Fourier coefficients of h . Specifically, since $\bar{T}_0 = \sqrt{\frac{2}{\pi}} T_0$ and $\bar{T}_k = \sqrt{\frac{1}{\pi}} T_k$, we have:

$$\sqrt{\frac{2}{\pi}} c_0 = \alpha_0, \quad \sqrt{\frac{1}{\pi}} c_k = \alpha_k \text{ for } k > 0. \quad (18)$$

With this fact in hand, Theorem B.6 follows almost immediately from Theorem B.5. Specifically, given $f \in \mathcal{F}([-1, 1], \mathbb{R})$ with Chebyshev series coefficients $c_0, c_1, \dots$, we let $g(\theta) = f(\cos \theta)$ and $h(\theta) = g(-|\theta|)$. Let $\alpha_0, \alpha_1, \dots$ denote h 's non-zero Fourier coefficients. Then, let $\tilde{h}$ be the approximation to h given by Theorem B.5. $\tilde{h}$ is a $2m - 2$ degree trigonometric polynomial and is even

since h is even and the bump function b is symmetric. Denote $\tilde{h}$'s non-zero Fourier coefficients by $\tilde{\alpha}_0, \dots, \tilde{\alpha}_{2m-2}$. We have that $\tilde{\alpha}_k = \frac{\tilde{b}_k}{b_0} \alpha_k$ for $0 \leq k \leq 2m-2$. Finally, let $\tilde{f} \in \mathcal{F}([-1, 1], \mathbb{R})$ be defined by $\tilde{f}(\cos(\theta)) = h(-\theta)$. By (18), we have that $\tilde{f}$ is a degree $2m-2$ polynomial and its Chebyshev series coefficients $\tilde{c}_0, \dots, \tilde{c}_{2m-2}$ are exactly equal to $\frac{\tilde{b}_k}{b_0} c_k$.

Moreover, we have $\|f - \tilde{f}\|_\infty = \max_{x \in [-1, 1]} |f(x) - \tilde{f}(x)| = \max_x |h(x) - \tilde{h}(x)|$. By Theorem B.5 we have $\max_x |h(x) - \tilde{h}(x)| < 9 \frac{\lambda}{m}$, so we conclude that $\|f - \tilde{f}\|_\infty < 9 \frac{\lambda}{m}$. $\square$

In addition to the main result of Theorem B.6, our SDE algorithm also require an additional property of the damped Chebyshev approximation $\tilde{f}$:

Lemma B.7. *For any non-negative function $f \in \mathcal{F}([-1, 1], \mathbb{R})$ (not necessarily Lipschitz), let $\tilde{f}$ be as in Theorem B.6. We have that $\tilde{f}$ is also non-negative on $[-1, 1]$.*

Proof. Let $h(\theta)$ and $\tilde{h}(\theta)$ be the 2π perioduc functions as in the proof of Theorem B.6. I.e., $h(\theta) = g(-|\theta|)$ where $g(\theta) = f(\cos \theta)$ and $\tilde{h}$ is the truncated, Jackson-damped approximation to h from Theorem B.5. If f is non-negative, then so is h , and since $\tilde{h}$ is the convolution of h with a non-negative function, it is non-negative as well. Finally, since $\tilde{f}(\cos(\theta)) = h(-\theta)$, we conclude that $\tilde{f}(x) \geq 0$ for $x \in [-1, 1]$. $\square$

C Derivation of Fact A.2

Let $x = \sin(u)$ then we have that $dx = \cos(u)du$. Substituting the change of variable in the integral and noting the fact that $T_k(\cos \theta) = \cos(k\theta)$ for $\theta \in [-\pi, \pi]$ gives us that

$$\begin{aligned} \int_a^b \frac{T_k(x)}{\sqrt{1-x^2}} dx &= \int_{\sin^{-1} a}^{\sin^{-1} b} \frac{\cos(k \cos^{-1} \sin(u))}{\sqrt{1-\sin^2(u)}} \cos(u) du = \int_{\sin^{-1} a}^{\sin^{-1} b} \cos(k(\pi/2 - u)) du \\ &= \frac{-\sin(k(\pi/2 - u))}{k} \Big|_{\sin^{-1} a}^{\sin^{-1} b} = \frac{-\cos(ku)}{k} \Big|_{\sin^{-1} a}^{\sin^{-1} b} \end{aligned}$$

where we used the fact that $\cos^2(u) + \sin^2(u) = 1$ and $\int \cos(u) du = \sin(u) + c$.